

04

Platform Engineering

Infrastructure as Code, GitOps & Internal Developer Platforms

IN THIS WEEK · 3 TOPICS · 47 SECTIONS

01

Infrastructure as Code (Terraform Advanced)

16 SECTIONS

- 01 Infrastructure as Code Principles
- 02 Declarative vs Imperative Infrastructure
- 03 Terraform Architecture Overview
- 04 Providers & Resources
- 05 Terraform State Management
- 06 Remote Backends & State Locking
- 07 Terraform Modules & Reusability
- 08 Variable & Output Management
- 09 Workspaces for Environment Isolation
- 10 Dependency Management in Terraform
- 11 DRY Infrastructure Patterns
- 12 Provisioners & When to Avoid Them
- 13 Terraform Plan & Apply Workflow
- 14 Infrastructure Versioning Strategy
- 15 Managing Multi-Environment Infrastructure
- 16 Terraform Security Best Practices

02

GitOps & Continuous Delivery

15 SECTIONS

- 01 GitOps Principles
- 02 Declarative Infrastructure & Deployments
- 03 ArgoCD Architecture
- 04 Helm Chart Fundamentals
- 05 Helm Templating & Values Management
- 06 Kustomize Basics
- 07 Application Deployment via GitOps
- 08 Automated Sync & Self-Healing
- 09 GitOps Repository Structure Design
- 10 Environment Promotion Strategy
- 11 Drift Detection & Reconciliation
- 12 Secret Management in GitOps
- 13 Rollback Strategies in GitOps
- 14 Multi-Cluster Deployment Patterns
- 15 Observability in GitOps Workflows

03

Internal Developer Platform (IDP)

16 SECTIONS

- 01 Platform Engineering Fundamentals
- 02 DevOps vs Platform Engineering
- 03 Internal Developer Platform (IDP) Concepts
- 04 Golden Path Strategy
- 05 Self-Service Infrastructure
- 06 Developer Experience (DevEx) Principles
- 07 Backstage Architecture Overview
- 08 Service Templates & Scaffolding
- 09 CI/CD Template Automation
- 10 Infrastructure Template Automation
- 11 Kubernetes Resource Templates
- 12 Policy as Code Concepts
- 13 Standardization & Governance
- 14 Platform API Design Concepts
- 15 Scaling Engineering Teams with IDP
- 16 Measuring Platform Success Metrics

01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Infrastructure as Code (Terraform Advanced)

01 Infrastructure as Code Principles

Let's start from the very beginning and understand why Infrastructure as Code exists, because once you understand the problem it solves, every other concept in this section makes intuitive sense.

Before Infrastructure as Code, provisioning infrastructure meant logging into a web console, clicking through menus, filling out forms, and manually configuring servers, networks, databases, and load balancers. This approach has a name — ClickOps — and it has serious, fundamental problems.

The first problem is reproducibility. If you manually configured a production environment over six months, clicking through hundreds of settings across dozens of services, can you reproduce that exact environment? Maybe. But it will take another six months, you'll forget things, you'll make slightly different choices, and your new environment will be subtly different from the original in ways that cause mysterious bugs. The environment exists in your hands and your memory, not in any durable, transferable form.

The second problem is collaboration. Your infrastructure configuration lives nowhere that your team can see, review, or contribute to. If you get hit by a bus tomorrow, nobody knows exactly how the infrastructure was configured. There's no version history, no code review, no way to understand why a particular setting was chosen.

The third problem is consistency across environments. You have development, staging, and production environments. They should be as similar as possible to ensure that what works in staging actually works in production. But with manual configuration, they inevitably drift apart. Someone changes a setting in production to fix an incident and forgets to make the same change in staging. Now they're different in ways nobody documented.

The fourth problem is speed and scale. Manually provisioning a new environment takes days or weeks. Running experiments that need temporary infrastructure means waiting for someone to set it up. Scaling out requires manually repeating the same steps across many resources.

Infrastructure as Code solves all of these by treating your infrastructure configuration the same way you treat application code. You write files that describe what infrastructure you want, store those files in version control, review changes through pull requests, and run automated tools that read your files and create the infrastructure they describe.

The core principles that define good Infrastructure as Code practice:

Everything is in version control — every piece of infrastructure configuration is checked into Git. Every change goes through a pull request. Every configuration has a commit history that answers who changed what, when, and why. If production is on fire, you can look at recent commits and immediately see what changed. You can revert to a previous state by reverting a commit. The infrastructure's history is as auditable as the application code's history.

Infrastructure is reproducible — given the same code, you should always get the same infrastructure. Running your Terraform code today and running it again in six months should produce identical infrastructure. This means no manual steps outside the code, no undocumented one-time configurations, no "oh, I also had to do this thing manually."

Infrastructure is idempotent — you can run your IaC tool multiple times against existing infrastructure without breaking anything. If the desired state matches the current state, nothing happens. If they differ, only the differences are applied. Running Terraform against infrastructure that's already in the right state makes no changes.

Environments are cattle, not pets — traditional servers were treated like pets: given names, carefully maintained, kept running for years. Infrastructure as Code environments are cattle: if one breaks, you don't fix it, you destroy it and create a new identical one from code. This is only possible because all configuration is in code — you're never afraid to delete something because you know exactly how to recreate it.

Self-documenting — your IaC code is the documentation of your infrastructure. Someone joining the team reads the Terraform files and understands the complete infrastructure topology, configurations, and relationships. No separate, inevitably outdated infrastructure documentation documents needed.

02 Declarative vs Imperative Infrastructure

This is a fundamental concept that underlies how Terraform and tools like it work, and understanding the difference deeply will make you a much better infrastructure engineer.

Imperative means you write instructions that describe the steps to achieve a result. You tell the computer exactly what to do, in what order. You are describing the process.

Here's an imperative approach to creating a server: "Check if a server named web-server-1 exists. If it doesn't exist, create it with these specifications. Then check if the security group exists. If it doesn't, create it. Then attach the security group to the server."

Notice that you have to think about the current state yourself. You have to check what exists, handle cases where things do or don't exist, and write different code paths for different scenarios. If you run this script twice and the server already exists from the first run, you need to handle that case explicitly — otherwise you might try to create a duplicate and fail.

Most traditional programming and scripting is imperative. Bash scripts that provision infrastructure are imperative. AWS CLI commands run in sequence are imperative. Ansible in its earlier form was largely imperative.

Declarative means you write a description of the desired end state. You tell the computer what you want to exist, not how to create it. The tool figures out the steps to get from the current state to your desired state.

Here's a declarative approach to the same server: "There should exist a server named web-server-1 with these specifications, attached to a security group with these rules."

That's it. You don't check if it exists first. You don't write different code for create vs update vs no-change scenarios. You just describe what you want. Terraform reads your description, figures out what currently exists, calculates the difference between current state and desired state, and executes only the steps needed to close that gap.

If the server doesn't exist, Terraform creates it. If it already exists with the right specs, Terraform does nothing. If it exists but with wrong specs, Terraform modifies only the specific attributes that are wrong. You don't think about any of this — you just write the desired state and Terraform handles the transition.

This has profound implications for how you write and reason about infrastructure.

Declarative code is self-healing — if someone manually changes infrastructure outside of Terraform (someone goes into the AWS console and changes a setting), your declarative code still describes what it should look like. The next time you run Terraform, it detects the drift and corrects it back to the declared state. Imperative scripts don't have this property — they don't know what currently exists, so they can't detect or correct drift.

Declarative code is easier to read and reason about — you can look at a Terraform file and immediately understand what the infrastructure looks like. It's a description of reality, not a series of steps to build reality. New team members can understand your infrastructure by reading the declarations, without needing to mentally trace through procedural logic.

The tradeoff — declarative systems can feel like magic because you don't control the steps. When something goes wrong, debugging requires understanding how the tool interprets your declarations, which can be less intuitive than debugging step-by-step procedural code. Understanding Terraform's plan output is essential for this reason — it's the window into how Terraform interpreted your declarations and what steps it plans to take.

Terraform is declarative. You declare desired state in HCL (HashiCorp Configuration Language). Terraform handles the imperative steps to get there.

03 Terraform Architecture Overview

Understanding how Terraform is actually structured internally helps you understand why it behaves the way it does, why certain operations are fast and others slow, and how to debug problems when they arise.

Terraform has several major components that work together.

Terraform Core is the brain of the operation. It's the binary you download and run. It reads your configuration files, builds an internal graph of all the resources you've declared and their relationships, calculates the difference between your desired state and the current state, and determines what operations need to happen and in what order. Core doesn't know anything about specific cloud providers or services — it doesn't know what an AWS EC2 instance or a Kubernetes namespace is. It only knows about the abstract concepts of resources, data sources, providers, and state.

Providers are plugins that extend Terraform Core with knowledge of specific platforms. The AWS provider teaches Terraform how to create, read, update, and delete AWS resources. The Kubernetes provider teaches it about Kubernetes resources. The Google Cloud provider, the Azure provider, the Vault provider, the Datadog provider — there are over a thousand providers in the public registry, and you can write custom private providers for internal platforms.

Providers are separate binaries from Terraform Core. When you run `terraform init`, Terraform downloads the provider plugins your configuration requires and stores them locally. Providers are versioned separately from Terraform Core, which means you can update Terraform Core independently from provider updates, and you can pin specific provider versions to ensure consistent behavior.

Resources are the actual infrastructure objects declared in your configuration — an EC2 instance, a VPC, a Kubernetes namespace, a Route53 DNS record. Each resource type is defined by a provider, which tells Terraform what attributes it has, which are required vs optional, and how to create/read/update/delete it through the provider's API.

Data Sources are how you read information about existing infrastructure that Terraform doesn't manage. If you have an existing VPC that was created outside of Terraform and you want to deploy resources into it, you use a data source to query information about that VPC. Data sources are read-only — they don't create or modify anything, they just fetch information.

State is Terraform's record of what infrastructure it has created and what the current attributes of that infrastructure are. This is a critical concept and we'll cover it in depth in its own section, but architecturally, state is how Terraform Core knows what already exists so it can calculate what needs to change.

The execution model — when you run `terraform plan`, here's what happens internally: Terraform reads all your `.tf` configuration files and builds a complete in-memory representation of your desired state. It reads the current state from the state file. It calls the relevant providers to refresh its knowledge of current resource attributes (checking if attributes have changed outside of Terraform). It builds a directed acyclic graph (DAG) of all resources and their dependencies, which determines the correct order of operations. It walks the graph, comparing desired state to current state for each resource, and produces a list of operations: create, update, or delete. This list of operations is the plan you review before applying.

When you run `terraform apply`, it executes those operations in the correct dependency order, calling the provider APIs to create, update, or delete each resource, and updates the state file to reflect the new reality after each operation.

04 Providers & Resources

Providers and resources are the vocabulary of Terraform. Once you understand how they work, reading and writing Terraform code becomes intuitive.

Providers are the adapters between Terraform and the outside world. Each provider is responsible for a specific API or platform. The AWS provider talks to the AWS API. The Kubernetes provider talks to the Kubernetes API. The GitHub provider talks to the GitHub API. The Cloudflare provider talks to Cloudflare's API.

In your Terraform configuration, you declare which providers you're using in a `required_providers` block, specifying the source (usually the Terraform Registry) and a version constraint:

hcl

13 lines

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "~> 2.0"
    }
  }
}
```

Then you configure each provider — telling it how to authenticate and what default settings to use:

hcl

4 lines

```
provider "aws" {  
  region = "us-east-1"  
}
```

Providers handle authentication to their respective APIs. The AWS provider can authenticate using environment variables (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY), IAM instance profiles, AWS SSO, or explicit credentials in the provider block. The Kubernetes provider can authenticate using a kubeconfig file, a service account token, or cluster certificate data.

Resources are the individual infrastructure objects you declare. Each resource has a type (defined by the provider), a local name (used to reference it within your Terraform code), and a body of configuration arguments.

hcl

10 lines

```
resource "aws_instance" "web_server" {  
  ami          = "ami-0c55b159cbfafa1f0"  
  instance_type = "t3.medium"  
  
  tags = {  
    Name          = "web-server"  
    Environment = "production"  
  }  
}
```

Here `aws_instance` is the resource type (provided by the AWS provider), `web_server` is the local name you've given it, and the block contains the configuration arguments for this specific instance.

Resource types follow a naming convention: the prefix matches the provider. `aws_instance`, `aws_s3_bucket`, `aws_vpc` are all AWS resources. `kubernetes_deployment`, `kubernetes_namespace`, `kubernetes_service` are all Kubernetes resources.

Resource attributes have two categories: arguments (inputs you provide to configure the resource) and attributes (values that are known only after the resource is created — like the ID or IP address that AWS assigns). You reference a resource's attributes in other resources using the syntax

`resource_type.local_name.attribute`:

hcl

4 lines

```
resource "aws_eip" "web_ip" {  
  instance = aws_instance.web_server.id # References the ID of the instance above  
}
```

This reference also creates an implicit dependency — Terraform knows it must create `aws_instance.web_server` before `aws_eip.web_ip` because the EIP needs the instance's ID.

Data sources look similar to resources but are prefixed with `data` :

hcl

10 lines

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical's AWS account

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-*--22.04-amd64-server-*"]
  }
}
```

This queries AWS for the most recent Ubuntu 22.04 AMI without creating anything. You then reference it as `data.aws_ami.ubuntu.id`.

05 Terraform State Management

State is one of the most important and most misunderstood aspects of Terraform. Getting state management wrong is the source of the majority of serious Terraform problems in production. Understanding it deeply is essential.

Terraform maintains a state file — by default named `terraform.tfstate` — that records what infrastructure Terraform has created and the current known attributes of that infrastructure. This file is Terraform's map of the real world. Without it, Terraform has no memory of what it has done.

Here's why state is necessary. Your Terraform configuration describes desired state. The real world has current state. To calculate what needs to change, Terraform needs to know both. The configuration file gives it desired state. The state file gives it a record of current state. Terraform compares the two and produces a plan.

Without state, Terraform would have no choice but to query every possible resource in your cloud account on every run to figure out what currently exists — which would be impossibly slow and still wouldn't reliably work because it couldn't know which resources in your account belong to this Terraform configuration vs resources created by other means.

What the state file contains — for every resource Terraform manages, the state file records: the resource type and local name, all the configuration arguments you provided, and all the attributes of that resource including ones assigned by the cloud provider after creation (like the AWS-assigned resource ID, IP addresses, ARNs). When you run `terraform plan`, Terraform compares the configuration to the state (and optionally refreshes the state by querying the provider API) to determine what has changed.

State file format — the state file is JSON. You can open it and read it. This is also a security concern — if your configuration includes sensitive values (passwords, keys), they end up in the state file in plain text. This is why securing access to your state file is critical.

State drift — drift happens when real infrastructure diverges from what's recorded in state. This can happen when someone modifies infrastructure manually outside of Terraform (going into the AWS console and changing a setting), when a resource changes state on its own (an EC2 instance is terminated by AWS due to a billing issue), or when the state file becomes inconsistent with reality for other reasons. Running `terraform refresh` updates the state file to reflect current reality without making any changes to infrastructure.

terraform state commands — Terraform provides a set of commands for directly manipulating state when necessary. `terraform state list` shows all resources tracked in state. `terraform state show resource_type.name` shows the state of a specific resource. `terraform state mv` moves a resource in state — used when you rename a resource in your configuration and need state to follow. `terraform state rm` removes a resource from state without destroying it — used when you want Terraform to stop managing a resource without deleting the actual infrastructure. `terraform import` adds an existing resource that was created outside Terraform into state so Terraform can manage it going forward.

These commands are powerful and should be used carefully. Corrupting state is one of the most painful situations in Terraform because it can cause Terraform to try to create resources that already exist or lose track of resources it should be managing.

06 Remote Backends & State Locking

By default, Terraform stores state in a local file. This works fine for individual experimentation but is completely unworkable for teams. If three engineers each have their own local state file, they have three divergent views of the infrastructure. If two engineers run `terraform apply` simultaneously, they'll corrupt each other's state. If a state file is lost or deleted, Terraform loses its map of the infrastructure.

Remote backends solve these problems by storing state in a shared, centralized location that all team members and CI/CD systems access.

Remote backends are storage systems where Terraform writes and reads its state file. The most commonly used backends are AWS S3, Google Cloud Storage, Azure Blob Storage, and Terraform Cloud. Instead of a local file, state lives in a cloud object store that all members of the team access.

Configuring a remote backend in Terraform looks like this:

hcl

10 lines

```
terraform {
  backend "s3" {
    bucket      = "my-company-terraform-state"
    key         = "production/ml-platform/terraform.tfstate"
    region      = "us-east-1"
    encrypt     = true
    dynamodb_table = "terraform-state-lock"
  }
}
```

This tells Terraform to store state in an S3 bucket at a specific path. The `encrypt = true` ensures the state file is encrypted at rest. The `dynamodb_table` reference is for state locking.

State locking is the mechanism that prevents two Terraform operations from running simultaneously against the same state, which would corrupt it. When you run `terraform apply`, Terraform first tries to acquire a lock on the state. If the lock is held by another process (another engineer or CI/CD run is in progress), Terraform waits or fails with an error telling you the lock is held and by whom. After the apply finishes, the lock is released.

For S3 backends, locking is implemented using a DynamoDB table. Terraform creates a record in DynamoDB when it acquires a lock and deletes it when done. DynamoDB's conditional writes ensure that only one process can acquire the lock at a time. For GCS backends, locking uses GCS object locking. For Terraform Cloud, locking is built into the service.

State file organization — as your infrastructure grows, you'll have state for many different components and environments. How you organize state files matters. The key principle: state should be scoped to what can change together and what has similar blast radius. If your networking infrastructure and your ML training infrastructure are in the same state file, then every Terraform operation that touches networking must have access to the full state including ML training resources. This creates unnecessary coupling.

Common organization patterns: one state file per environment per component — `production/networking/terraform.tfstate`, `production/ml-platform/terraform.tfstate`, `staging/networking/terraform.tfstate`. This scopes blast radius and access control — the team managing ML infrastructure doesn't need access to the networking state and can't accidentally destroy networking resources.

State file security — because state contains sensitive values in plain text, the S3 bucket or GCS bucket storing state must have strict access controls. Only the engineers and CI/CD systems that legitimately need to manage that infrastructure should have read/write access. Enable bucket versioning so you can recover from accidental state corruption. Enable server-side encryption. Block public access completely.

07 Terraform Modules & Reusability

As your Terraform codebase grows, you'll find yourself writing the same patterns over and over. Every time you create a new ML training environment, you create a VPC, subnets, security groups, IAM roles, EFS for shared storage, an EKS cluster, node groups, and a set of Kubernetes namespaces — in the same pattern, with the same relationships, but with different names and slightly different sizes. Writing all of that from scratch every time is tedious, error-prone, and means any improvement to the pattern needs to be applied in dozens of places.

Modules are Terraform's mechanism for abstraction and reuse. A module is a collection of Terraform configuration files in a directory that encapsulate a pattern, expose inputs, and produce outputs. You write the pattern once and call it multiple times with different inputs.

Think of a Terraform module like a function in programming. A function has parameters (inputs), a body of logic, and a return value (outputs). A module has input variables (the parameters), a set of resources and data sources (the body), and output values (the return values).

Here's what a simple module structure looks like. You create a directory called `modules/ml-training-cluster/` and inside it you have:

`main.tf` — all the resources the module creates: EKS cluster, node groups, IAM roles, security groups.

`variables.tf` — the inputs the module accepts:

```
variable "environment" {
  description = "Environment name (dev, staging, production)"
  type        = string
}

variable "cluster_name" {
  description = "Name for the EKS cluster"
  type        = string
}

variable "node_count" {
  description = "Number of worker nodes"
  type        = number
  default     = 3
}
```

`outputs.tf` — the values the module exposes after creation:

```
output "cluster_endpoint" {
  description = "EKS cluster API endpoint"
  value       = aws_eks_cluster.main.endpoint
}

output "cluster_name" {
  description = "Name of the created cluster"
  value       = aws_eks_cluster.main.name
}
```

Then you call this module from your root configuration:

```
module "production_cluster" {
  source      = "./modules/ml-training-cluster"
  environment = "production"
  cluster_name = "ml-prod"
  node_count  = 10
}

module "staging_cluster" {
  source      = "./modules/ml-training-cluster"
  environment = "staging"
  cluster_name = "ml-staging"
  node_count  = 3
}
```

Two clusters, completely different sizes and names, from the same module definition. The underlying pattern — how the cluster is structured, what IAM roles it gets, what security groups control its access — is defined once and reused.

Module sources — modules can come from different places. Local directories (as above), the Terraform public registry ([hashicorp/kubernetes/aws](https://registry.terraform.io/modules/hashicorp/kubernetes/aws)), a private registry, Git repositories, or S3 buckets. Using modules from the public registry gives you community-maintained, well-tested implementations of common patterns. Using private Git repositories lets you share modules across your organization without making them public.

Module versioning — when you use a module from a Git source, you should pin it to a specific version tag:

```
module "ml_cluster" {
  source = "git::https://github.com/myorg/terraform-modules.git//ml-cluster?ref=v2.3.0"
}
```

Pinning versions ensures that a module update doesn't unexpectedly change all the infrastructure that uses it. You update intentionally by changing the version reference, testing in staging first.

Nested modules — modules can call other modules. A high-level [ml-environment](#) module might call a [networking](#) module, a [compute-cluster](#) module, and a [storage](#) module, composing them into a complete environment.

08 Variable & Output Management

Variables and outputs are the interface of your Terraform configuration and modules. Variables are inputs — things you provide to configure what gets built. Outputs are things your configuration exposes after building — values that other configurations or humans need to know about.

Input Variables are declared with a [variable](#) block:

```
variable "instance_type" {
  description = "EC2 instance type for model serving"
  type        = string
  default     = "g4dn.xlarge"

  validation {
    condition     = contains(["g4dn.xlarge", "g4dn.2xlarge", "p3.2xlarge"], var.instance_type)
    error_message = "Instance type must be a GPU instance type."
  }
}
```

Several things to note here. The `description` is not optional in good Terraform code — it documents what the variable does. The `type` constraint ensures the variable receives the right kind of value — string, number, bool, list, map, or complex object types. The `default` makes the variable optional — if no value is provided, the default is used. The `validation` block enforces business rules on the input — here ensuring only GPU instance types are used for model serving.

How variable values are provided — in order of precedence (highest wins): command line flags (`terraform apply -var="instance_type=p3.2xlarge"`), `.tfvars` files (files containing variable assignments), environment variables prefixed with `TF_VAR_` (`TF_VAR_instance_type=p3.2xlarge`), and default values in the variable declaration. In practice, you use `.tfvars` files for environment-specific values and default values for sensible defaults.

tfvars files — you typically have one per environment. `production.tfvars` contains production-specific values (larger instance types, higher replica counts, production database sizes). `staging.tfvars` contains staging-specific values. You pass the appropriate file when running Terraform: `terraform apply -var-file=production.tfvars`.

Sensitive variables — mark variables that contain secrets with `sensitive = true`:

```
variable "database_password" {
  description = "Master password for the RDS instance"
  type        = string
  sensitive   = true
}
```

This prevents Terraform from displaying the value in plan output or logs. It doesn't prevent the value from being stored in state (it still is, in plain text), but it prevents accidental exposure in terminal output.

Output Values are declared with an `output` block:

```
output "model_serving_endpoint" {
  description = "URL for the model serving API"
  value      = "https://${aws_lb.model_serving.dns_name}/predict"
}

output "kubeconfig_command" {
  description = "Command to configure kubectl access to the cluster"
  value      = "aws eks update-kubeconfig --name ${aws_eks_cluster.main.name} --region ${var.aws_region}"
}
```

Outputs serve several purposes. They expose information from a module to the calling configuration. They display useful information to the person running Terraform (like how to connect to the infrastructure just created). They allow cross-module references — one module can use another module's outputs as inputs.

Complex variable types — for ML infrastructure, you often want to pass structured configuration rather than individual primitive values:

```
variable "node_groups" {
  description = "Configuration for EKS node groups"
  type = map(object({
    instance_type = string
    min_size      = number
    max_size      = number
    desired_size  = number
    gpu_enabled   = bool
  }))
}
```

This lets callers pass a map of node group configurations, enabling flexible, composable infrastructure definitions.

09 Workspaces for Environment Isolation

Terraform workspaces are a built-in mechanism for maintaining multiple independent state files from the same Terraform configuration. The idea is that you have one configuration and you create separate workspaces for different environments (dev, staging, production), each with its own state.

By default, every Terraform configuration operates in a workspace called `default`. You create additional workspaces with `terraform workspace new staging` and switch between them with `terraform workspace select production`. The current workspace name is accessible in your configuration via `terraform.workspace`, which lets you use it to vary behavior by environment.

```
locals {
  instance_type = {
    dev      = "t3.medium"
    staging   = "g4dn.xlarge"
    production = "g4dn.2xlarge"
  }
}

resource "aws_instance" "model_server" {
  instance_type = local.instance_type[terraform.workspace]
  # ...
}
```

This configuration automatically uses the right instance type based on which workspace is active.

Workspaces are appealing for their simplicity — one configuration, multiple environments. However, it's important to understand their limitations so you can decide when they're the right tool and when they're not.

Where workspaces work well — for genuinely identical environments that differ only in scale (same topology, different sizes), workspaces are clean and simple. Feature branches or ephemeral environments for testing — you create a workspace for a PR, test the infrastructure changes, then delete the workspace and the state is gone.

Where workspaces fall short — for environments that differ structurally, not just in scale. Production might have a multi-region setup, multi-AZ databases, WAF configuration, and compliance controls that staging doesn't have. Trying to express all of this through workspace-based conditionals (`if terraform.workspace == "production"`) leads to messy, hard-to-read configuration. At that point, separate configurations per environment (the directory-per-environment pattern) is cleaner.

Workspace vs directory-per-environment — the alternative to workspaces is maintaining separate directories for each environment, each with its own backend configuration pointing to separate state. `environments/production/main.tf`, `environments/staging/main.tf`. These share modules but have completely independent configurations and state. This is more verbose but also more explicit and harder to accidentally apply production changes to staging. For production-grade, long-running environments, directory-per-environment is generally preferred. Workspaces are great for ephemeral or highly similar environments.

10 Dependency Management in Terraform

Terraform builds a directed acyclic graph (DAG) of all resources in your configuration and uses this graph to determine the correct order of operations and which resources can be created in parallel. Understanding how dependencies work helps you write more correct and more efficient Terraform.

Implicit dependencies are created automatically when you reference one resource's attributes in another resource's configuration. When you write:

```
resource "aws_subnet" "ml_subnet" {  
  vpc_id = aws_vpc.main.id # Reference to aws_vpc.main  
  # ...  
}
```

Terraform automatically knows that `aws_vpc.main` must be created before `aws_subnet.ml_subnet` because the subnet needs the VPC's ID. This is an implicit dependency — you didn't explicitly declare it, but Terraform inferred it from the reference. This is the preferred way to express dependencies because it's directly tied to the actual data relationship.

Explicit dependencies with `depends_on` are for cases where there's a dependency that isn't expressed through attribute references. This is less common but necessary sometimes:

```
resource "aws_iam_role_policy_attachment" "ml_training" {  
  role      = aws_iam_role.training.name  
  policy_arn = aws_iam_policy.ml_access.arn  
}  
  
resource "aws_eks_node_group" "training_nodes" {  
  cluster_name = aws_eks_cluster.main.name  
  # ...  
  
  depends_on = [  
    aws_iam_role_policy_attachment.ml_training # Must exist before nodes can start  
  ]  
}
```

Here the node group doesn't reference the policy attachment's attributes directly, but the nodes won't function correctly until the IAM policy is attached to the role. The `depends_on` makes this implicit requirement explicit.

Resource dependencies in practice — the typical dependency chain for an ML infrastructure might look like: VPC → Subnets → Security Groups → EKS Cluster → Node Groups → Kubernetes Namespaces → Helm Releases → Kubernetes ConfigMaps. Each layer depends on the layer below it. Terraform's DAG handles this automatically through the implicit references you create when writing the configuration.

Parallel creation — resources with no dependency relationship between them are created in parallel. Terraform determines from the DAG which resources have no ordering constraint and runs their API calls concurrently. This is why a large Terraform apply that creates hundreds of resources completes much faster than sequential creation would — independent resources are created simultaneously.

The `-parallelism` flag controls how many concurrent operations Terraform runs. Default is 10. For large infrastructures, increasing this speeds up applies. For APIs with strict rate limits, decreasing it prevents rate limiting errors.

Circular dependencies — if resource A depends on resource B and resource B depends on resource A, Terraform detects this as a circular dependency and refuses to create a plan. Circular dependencies are always a design problem — they mean your resource relationships are fundamentally tangled. The solution is to restructure the design to break the cycle, often by introducing an intermediate resource or reordering how attributes are referenced.

11 DRY Infrastructure Patterns

DRY stands for Don't Repeat Yourself. In the context of Terraform, it means structuring your code so that the same concept, pattern, or configuration exists in exactly one place. If you need to change it, you change it in one place and the change propagates everywhere it's used.

Repeated code in Terraform is dangerous in ways that are specific to infrastructure. If you have the same security group rule defined in 15 different places across your codebase, and a security team requirement changes that rule, you need to find and update all 15 instances. Miss one and you have an inconsistent security posture that might only be discovered in an audit or after an incident.

Modules are the primary DRY mechanism — as discussed in the modules section, encapsulating a repeated infrastructure pattern into a module and calling it with different inputs is the primary way to eliminate repetition.

Local values are another DRY tool within a single configuration. Instead of repeating the same expression or the same set of tags in every resource:

hcl

24 lines

```
locals {
  common_tags = {
    team      = "ml-platform"
    managed_by = "terraform"
    environment = var.environment
    cost_center = "ML-Infrastructure"
  }

  name_prefix = "${var.environment}-${var.project_name}"
}

resource "aws_instance" "training_node" {
  # ...
  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-training"
    Role = "model-training"
  })
}

resource "aws_s3_bucket" "model_artifacts" {
  bucket = "${local.name_prefix}-model-artifacts"
```

```
tags    = local.common_tags
}
```

The common tags are defined once in `locals`. Every resource uses them. If you need to add a new tag organization-wide, you add it in one place.

for_each and count for resource repetition — when you need multiple instances of a resource with similar but slightly different configurations, instead of copying the resource block multiple times, use `for_each` or `count`:

hcl

20 lines

```
variable "ml_namespaces" {
  default = {
    training = { cpu_limit = "16", memory_limit = "64Gi" }
    serving  = { cpu_limit = "8",  memory_limit = "32Gi" }
    feature  = { cpu_limit = "4",  memory_limit = "16Gi" }
  }
}

resource "kubernetes_namespace" "ml" {
  for_each = var.ml_namespaces

  metadata {
    name = each.key
    labels = {
      environment = var.environment
      purpose      = each.key
    }
  }
}
```

One resource block creates three namespaces with different names and can be extended by adding entries to the `ml_namespaces` variable — no code duplication required.

Composition over inheritance — rather than trying to build one mega-module that handles every possible configuration through a complex web of conditionals, build small, focused modules that each do one thing well, and compose them together. This is DRY at the architectural level — the same small module is used in many places.

The tradeoff of abstraction — DRY is not about eliminating all repetition regardless of cost. Over-abstraction makes code harder to understand and modify. If you abstract a pattern that only appears twice and might be different in the future, you've created complexity without sufficient benefit. Apply DRY where the repetition is real, substantial, and likely to need to change together.

12 Provisioners & When to Avoid Them

Provisioners in Terraform are mechanisms for executing scripts or commands on a resource after it's created — running a shell script on a newly created EC2 instance, copying files to it, or executing a remote configuration management command against it.

Terraform supports three types: `local-exec` runs a command on the machine where Terraform is executing. `remote-exec` connects to a remote resource (usually via SSH or WinRM) and runs commands on it. `file` copies files from the machine running Terraform to a remote resource.

Before explaining when to avoid them, let me explain when they seem useful, because that makes the pitfalls obvious.

Suppose you create an EC2 instance with Terraform and then need to install some software on it before it's usable. The naive approach is to add a `remote-exec` provisioner:

hcl

20 lines

```
resource "aws_instance" "training_server" {
  ami          = data.aws_ami.ubuntu.id
  instance_type = "p3.2xlarge"

  provisioner "remote-exec" {
    inline = [
      "sudo apt-get update",
      "sudo apt-get install -y nvidia-driver-470",
      "sudo apt-get install -y python3-pip",
      "pip3 install torch transformers",
    ]

    connection {
      type      = "ssh"
      user      = "ubuntu"
      private_key = file("~/ssh/id_rsa")
    }
  }
}
```

This seems to work. But here's why it's problematic.

Provisioners break idempotency — if the provisioner script fails halfway through (network interruption, package server unavailable), Terraform marks the resource as "tainted" and will destroy and recreate it on the next apply. This means your EC2 instance gets destroyed and a new one created every time the provisioner fails. This is destructive and unpredictable.

Provisioners break the declarative model — the scripts you run in provisioners are imperative. They change state in ways that aren't tracked in Terraform's state file. The next time you run `terraform plan`, Terraform won't know that the software was installed and won't include it in its desired state calculation. If someone terminates and recreates the instance, the provisioner runs again — but if something has changed about the environment, the provisioner might fail or produce different results.

Provisioners require network access and credentials — for `remote-exec`, Terraform needs SSH access to the instance during apply. This requires your CI/CD system to have SSH keys and network access to the instance, which creates operational complexity and security concerns.

Provisioners are slow and fragile — downloading and installing packages at instance creation time is slow and dependent on external services being available. A flaky package mirror fails your entire Terraform apply.

What to use instead:

For instance configuration, use cloud-init user data (baked into the instance's launch configuration and runs on first boot without Terraform maintaining a connection) or pre-baked AMIs (build an AMI with all software pre-installed using Packer, then have Terraform launch instances from the pre-baked AMI — fast, reliable, fully versioned). For ongoing configuration management, use dedicated tools like Ansible, Chef, or Puppet that are designed for this and handle drift detection, idempotency, and failure recovery properly.

When provisioners are acceptable — they're not categorically forbidden. Local-exec is sometimes used for integrations where no better mechanism exists, like calling an external registration API after creating a resource. File provisioner can be useful in specific bootstrap scenarios. But they should always be a last resort, and you should be able to explain clearly why better alternatives aren't applicable.

13 Terraform Plan & Apply Workflow

The plan and apply workflow is the core operational loop of Terraform. Understanding what happens at each stage and how to use this workflow correctly is fundamental to operating Terraform safely in production.

`terraform init` is the first command you run in any Terraform directory. It initializes the working directory: downloads provider plugins specified in your configuration, downloads any modules referenced from remote sources, and configures the backend (connecting to the remote state). You must run init whenever you add a new provider or module, change provider versions, or change the backend configuration. Init is safe to run repeatedly — it's idempotent and doesn't modify infrastructure.

`terraform validate` checks that your configuration is syntactically valid and internally consistent — that all required arguments are provided, that references to other resources are valid, that types match. It doesn't connect to any providers or check whether resources actually exist. It's fast and safe to run as part of a CI check on every PR.

`terraform plan` is where the real work happens. Terraform reads your configuration, reads the current state, optionally calls provider APIs to refresh resource attributes, builds the dependency graph, compares desired state to current state for every resource, and produces a human-readable diff of what it intends to do. Create (green +), update in place (yellow ~), destroy (red -), and destroy then recreate (-/+) are the four operation types.

Reading the plan output carefully is the most important habit you can develop as a Terraform practitioner. Before running apply, you should be able to answer: How many resources are being created, updated, or destroyed? For updates, which specific attributes are changing and are those the changes you expect? Are there any unexpected destroys? A `-/+` (destroy and recreate) is particularly important to notice — it means the resource will be deleted and recreated, causing potential downtime and loss of any data associated with the resource.

You should save the plan to a file for important applies: `terraform plan -out=plan.tfplan`. This ensures that what you apply is exactly what was planned — no changes can sneak in between planning and applying.

`terraform apply` executes the planned changes. If you run it without a saved plan file, it will re-plan first and show you the plan, then ask for confirmation before proceeding. If you run it with a saved plan file (`terraform apply plan.tfplan`), it executes exactly the operations in the plan without re-planning or asking for confirmation. The latter is the preferred pattern for CI/CD pipelines — plan in one stage, review, then apply in a subsequent stage.

During apply, Terraform executes resource operations in the order determined by the dependency graph, running independent operations in parallel. It updates the state file after each resource operation succeeds — if the apply fails partway through, the state reflects what was successfully completed, so a subsequent apply will continue from where it left off rather than starting from scratch.

`terraform destroy` plans and applies the destruction of all resources managed in the current state. This is a dangerous operation and should be used very carefully in production. For destroying specific resources rather than everything, use `terraform destroy -target=resource_type.name`.

The `-target` flag applies changes to only a specific resource and its dependencies, ignoring everything else. This is useful for debugging or for making targeted changes in emergencies. However, using `-target` regularly is a code smell — it suggests your configuration isn't properly modularized and you're trying to work around unintended dependencies.

14 Infrastructure Versioning Strategy

Versioning your infrastructure code is not just about storing it in Git — it's about deliberately managing changes to infrastructure over time so that any version can be understood, reproduced, and rolled back if needed.

Semantic versioning for modules — when you build reusable Terraform modules, version them using semantic versioning. Patch versions (1.0.0 → 1.0.1) for bug fixes that don't change the interface. Minor versions (1.0.0 → 1.1.0) for new features that are backward compatible. Major versions (1.0.0 → 2.0.0) for breaking changes — changes that will require callers to update their configurations. Tag your Git repository with version tags and reference those tags in module sources.

Pin provider versions — always pin provider versions in your `required_providers` block. Using `version = "≥ 5.0"` (meaning `>= 5.0.0` and `< 6.0.0`) prevents major version upgrades from breaking your code while allowing patch updates. Using no version constraint at all means a provider update could introduce breaking changes at any time. Update provider versions deliberately, test in staging first, and document the update in a commit message.

Terraform version pinning — pin the Terraform version itself using a `.terraform-version` file (used by `tfenv`, the Terraform version manager) or in the `required_version` field in the `terraform` block. Different versions of Terraform have different behaviors and syntax, and using different versions across team members or CI/CD creates inconsistencies.

Infrastructure change management — large infrastructure changes should go through the same process as application changes. Write the change in a branch, get it reviewed via pull request, run `terraform plan` in CI and include the plan output in the PR for reviewers to see exactly what will change, merge and deploy. A plan output in a PR is tremendously valuable for reviewers because it shows the concrete effect of the code change on real infrastructure.

Tagging infrastructure for traceability — every resource Terraform creates should be tagged with information that links it back to its code origin. At minimum: the Terraform module or configuration that manages it, the environment it belongs to, the team responsible for it, the Git repository and commit that last modified it. This makes it trivially easy to find the code for any resource you're looking at in the cloud console, and makes cost allocation possible.

State file versioning — enable versioning on your S3 bucket (or whatever storage backs your remote state). If state becomes corrupted, S3 versioning lets you roll back to a previous state version. This has saved many teams from potentially catastrophic situations where state corruption made Terraform believe resources needed to be destroyed.

15 Managing Multi-Environment Infrastructure

Managing multiple environments — typically development, staging, and production — is where most of the complexity and best practice nuance in Terraform lives. Get this right and your infrastructure operations are smooth and safe. Get it wrong and you're constantly fighting environment drift, accidental production changes, and configuration confusion.

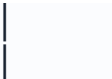
The core tension in multi-environment Terraform is this: you want environments to be similar enough that staging validations are meaningful for production, but you also need environments to be different enough that they're cost-appropriate (development doesn't need production-scale resources) and appropriately isolated (a staging mistake doesn't affect production).

Directory-per-environment structure is the most explicit and safest approach for production-grade infrastructure. You organize your Terraform code like this:

diagram

22 lines

```
infrastructure/
├── modules/
│   ├── ml-cluster/
│   ├── networking/
│   └── model-registry/
├── environments/
│   ├── development/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── terraform.tfvars
│   │   └── backend.tf
│   ├── staging/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── terraform.tfvars
│   │   └── backend.tf
│   └── production/
│       ├── main.tf
│       └── variables.tf
```



```
├─ terraform.tfvars
└─ backend.tf
```

Each environment directory has its own `main.tf` that calls shared modules with environment-appropriate inputs, its own `backend.tf` pointing to environment-specific state storage, and its own `terraform.tfvars` with environment-specific variable values. Each environment has completely independent state, independent backend, and independent apply workflow.

Variable differentiation — the `terraform.tfvars` files capture all the differences between environments. Production gets `node_count = 20`, `instance_type = "p3.2xlarge"`, `enable_multi_az = true`. Staging gets `node_count = 5`, `instance_type = "g4dn.xlarge"`, `enable_multi_az = false`. Development gets `node_count = 2`, `instance_type = "t3.large"`, `enable_multi_az = false`. All three environments use the same modules and patterns, just with different scale.

Promotion workflow — changes should always flow from development → staging → production, never directly to production. Your CI/CD pipeline enforces this. When a PR is merged to the main branch, the CI system automatically runs `terraform plan` against all environments and may auto-apply to development. Staging requires a manual approval or a specific trigger. Production requires a more formal approval process, possibly with a mandatory waiting period after staging validation.

Account isolation — for serious production environments, consider isolating environments in completely separate cloud accounts, not just separate state files in the same account. Separate AWS accounts provide a much harder security boundary — a compromised IAM role in staging cannot access production resources in another account. AWS Organizations makes managing multiple accounts tractable. The Terraform code itself stays the same; you just configure different provider credentials for each account.

16 Terraform Security Best Practices

Terraform's power — it can create, modify, and destroy production infrastructure — is also what makes it a high-value target for security mistakes. Insecure Terraform usage can expose credentials, grant excessive permissions, or allow unauthorized changes to critical infrastructure.

Never hardcode credentials in Terraform files — this is the most commonly violated rule and the most dangerous. AWS access keys, database passwords, API tokens should never appear in your `.tf` files or `.tfvars` files. These files go into version control, and secrets in version control are exposed to everyone with read access to the repository, forever. Use environment variables, AWS IAM roles with instance profiles or OIDC federation, or a secrets manager to inject credentials at runtime.

Protect your state file — as covered earlier, state files contain sensitive values in plain text. Your state storage backend must have access controls allowing only authorized humans and systems to read and write state. Enable encryption at rest. Enable versioning for recovery. Never commit state files to version control — add `*.tfstate` and `*.tfstate.backup` to your `.gitignore`.

Least privilege for Terraform's IAM permissions — the IAM user or role that Terraform uses to create infrastructure should have exactly the permissions needed to manage the resources in its scope, and nothing more. Terraform doesn't need to be able to access your billing information, your organization's root account, or services it doesn't manage. Scope permissions to specific resources and resource types wherever IAM allows. For production applies, consider using short-lived credentials via OIDC federation rather than long-lived IAM user access keys.

CI/CD pipeline security — your CI/CD pipeline that runs Terraform applies is extremely sensitive. If an attacker can inject malicious Terraform code into a branch that then gets planned and applied, they can create resources under your cloud account or exfiltrate data. Protect the CI/CD system: require PR reviews before apply, use OIDC federation for temporary credentials rather than static keys stored as CI secrets, run plans in untrusted contexts but applies only from protected branches.

Sentinel and OPA for policy enforcement — beyond access control, enforce infrastructure standards as code. You can use OPA or Terraform's Sentinel (in Terraform Cloud/Enterprise) to automatically check plans against organizational policies before they're applied. Policies like "no security groups with 0.0.0.0/0 ingress on port 22," "all S3 buckets must have encryption enabled," "no public RDS instances" — these are checked automatically and block non-compliant plans. This is safer than relying on human reviewers to catch every policy violation in PR reviews.

terraform plan review in CI — your CI pipeline should run `terraform plan` on every PR and post the plan output as a PR comment. Reviewers must look at the plan, not just the code. Code changes that look small can produce large infrastructure changes — reviewers need to see what actually changes. A plan showing `-/+ destroy and recreate` for a production database should raise an immediate flag.

Audit logging for Terraform applies — every `terraform apply` against production should be logged — who ran it, when, what plan was applied, what the outcome was. In CI/CD environments, this is captured in pipeline logs. For human applies, use a wrapper script that logs applies to a centralized system. Combined with state file versioning, this gives you a complete audit trail of every infrastructure change.

Scanning Terraform code for security issues — static analysis tools like `tfsec`, `checkov`, and `Terrascan` analyze your Terraform code for security misconfigurations before you apply anything. They catch common mistakes: S3 buckets without logging enabled, security groups too permissive, unencrypted EBS volumes, public-facing resources that should be private. Run these in your CI pipeline alongside `terraform validate`.

The thread that runs through all of them is this: infrastructure is code, and everything that makes application code better — version control, testing, code review, modularity, documentation, least privilege — applies equally to infrastructure code. Terraform is the tool, but the discipline is treating infrastructure with the same rigor and care you'd give to any production software system.

GitOps & Continuous Delivery

01 GitOps Principles

To understand GitOps, you first need to understand the problem it solves, because the solution only makes sense in the context of the problem.

Before GitOps, deploying software to Kubernetes was done imperatively. A developer or CI/CD pipeline would run `kubectl apply`, `helm upgrade`, or a series of cloud CLI commands to push changes to a cluster. This worked but created serious operational problems. There was no single record of what was actually deployed — the cluster's state existed only in the cluster itself and in people's heads. Anyone with cluster access could make changes directly, and those changes would leave no trace in version control. If something went wrong, figuring out what changed, when, and who changed it required digging through cluster audit logs that weren't always available or readable. Recreating a cluster after a disaster meant reconstructing dozens of manual steps from memory or incomplete documentation.

GitOps is an operational framework that uses Git repositories as the single source of truth for both infrastructure and application configuration, and uses automated agents to continuously ensure the running system matches what Git says it should look like.

The four core principles of GitOps, defined by Weaveworks who coined the term:

Declarative — the entire system is described declaratively. You don't write scripts that say "if this resource doesn't exist, create it; if it exists but is wrong, update it." You write declarations that say "this resource should look like this." The tool figures out how to make reality match your declaration. Kubernetes manifests, Helm charts, and Terraform files are all declarative. Shell scripts with imperative `kubectl` commands are not GitOps.

Versioned and Immutable — the desired state is stored in Git, which is an immutable, append-only version history. You cannot go back and change what was deployed at 2pm last Tuesday — that commit exists forever. Every change is a new commit. This gives you a complete, tamper-evident history of every change to your system. Who changed what, when, why (from the commit message), and what it changed from (the diff). This is the audit trail that compliance teams dream of and on-call engineers need during incidents.

Pulled automatically — the desired state is automatically applied to the system by an agent running inside the system, not pushed from outside. This is the pull model. An agent (ArgoCD, Flux) running in your Kubernetes cluster watches your Git repository. When it detects a difference between what Git says should exist and what actually exists in the cluster, it automatically reconciles — applying the Git state to the cluster. This is different from CI/CD pipelines that push changes from outside the cluster, which requires the external system to have cluster credentials.

Continuously reconciled — the agent doesn't just apply changes once when you push to Git. It continuously watches both Git and the cluster, ensuring they stay in sync. If someone manually changes a resource in the cluster (a common problem in conventional setups), the agent detects the drift and reverts it back to the Git-declared state. The cluster is self-healing — any unauthorized or accidental modification is automatically corrected.

The practical implications of these principles working together: your Git repository is always an accurate representation of what's running in production. Deploying a new version of an application is as simple as updating a value in a YAML file and pushing to Git. Recovering from a disaster is as simple as pointing a new cluster at the same Git repository. Security is improved because cluster credentials don't need to leave the cluster — the agent has them internally. And your entire deployment history is your Git history.

GitOps doesn't eliminate CI/CD — it changes where the CD part happens. CI (build, test, push image) remains a traditional push-based pipeline. CD moves to the pull-based GitOps model where an agent reconciles cluster state with Git state continuously, rather than a pipeline pushing changes to the cluster.

02 Declarative Infrastructure & Deployments

Declarative is a word thrown around a lot in modern infrastructure conversations, and it's worth being precise about what it means and why it matters so much to GitOps.

Declarative means describing the desired end state without specifying the steps to get there. You say what you want, not how to achieve it.

Imperative means specifying a sequence of steps to execute. You say what to do, step by step.

A concrete comparison. You want a Kubernetes Deployment with 5 replicas running nginx:

Imperative approach:

bash

3 lines

```
kubectl create deployment nginx --image=nginx
kubectl scale deployment nginx --replicas=5
```

Declarative approach:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.25.0
```

Applied with `kubectl apply -f deployment.yaml`.

The superficial difference looks minor. The practical difference is enormous.

Run the imperative commands twice and you'll try to create a deployment that already exists, fail on the create, and possibly misconfigure the scale. The commands are not idempotent — running them multiple times produces different behavior than running them once.

Apply the declarative manifest twice and nothing happens the second time — Kubernetes sees the desired state matches actual state and makes no changes. Declarative specifications are idempotent by nature.

Change the imperative commands to update the deployment — you need to write new commands that account for the current state. If the current replica count is 5 and you want 3, you run a scale command. If you don't know the current state, you can't write the right commands.

Change the declarative manifest to update the deployment — you change `replicas: 5` to `replicas: 3` in the YAML file and apply it. You don't need to know the current state. You just declare what you want.

Drift detection with declarative — because you've declared the desired state explicitly, any deviation from that state is detectable. If someone manually scales the deployment to 7 replicas, a tool watching the manifest can detect "actual state (7 replicas) differs from desired state (5 replicas)" and automatically correct it. With imperative scripts, there's no declared desired state to compare against.

Self-documentation — a declarative manifest is complete documentation of the resource's configuration. Reading the YAML file tells you exactly what the resource looks like. Imperative scripts tell you what commands were run, not what state was produced.

In GitOps, everything must be declarative because the entire model depends on Git containing the complete desired state of the system. If parts of your system are configured imperatively (someone ran `kubectl` commands that aren't captured anywhere), GitOps can't manage those parts — it doesn't know what they should look like.

This is why Kubernetes YAML manifests, Helm charts, and Kustomize overlays are the building blocks of GitOps. They're all declarative specifications of desired state that GitOps agents can compare against actual state and reconcile when they differ.

03 ArgoCD Architecture

ArgoCD is the most widely adopted GitOps operator for Kubernetes. It runs inside your Kubernetes cluster, watches one or more Git repositories, and continuously ensures your cluster matches the state declared in those repositories. Understanding ArgoCD's architecture helps you configure it correctly, troubleshoot problems, and understand why it makes the design choices it does.

ArgoCD is itself a set of Kubernetes deployments and services. When you install ArgoCD (typically via a Helm chart or the official install manifest), it creates a namespace with several components, each responsible for a distinct piece of the GitOps workflow.

API Server is ArgoCD's control plane. It exposes a REST API and gRPC API that the ArgoCD UI, the ArgoCD CLI, and other tools use to interact with ArgoCD. It handles authentication (integrated with OIDC/SSO, Dex for identity federation), authorization (RBAC policies controlling who can sync which applications), and CRUD operations on Application and AppProject objects. The API server is the only ArgoCD component that has an external-facing service — everything else communicates internally.

Repository Server is responsible for cloning Git repositories and rendering Kubernetes manifests. When ArgoCD needs to know what a Git repository declares for an application, the Repository Server clones the repo (or uses a local cache), runs whatever rendering tool is needed (plain YAML, Helm template, Kustomize build, Jsonnet), and returns the resulting Kubernetes manifests. The Repository Server is stateless — it doesn't store anything permanently — but it caches repository contents to avoid re-cloning on every reconciliation cycle.

Application Controller is the heart of ArgoCD. It's a Kubernetes controller (implementing the standard controller reconciliation loop pattern) that watches Application objects. For each Application, it periodically compares the desired state (what the Repository Server rendered from Git) against the live state (what the Kubernetes API says currently exists). If they differ and auto-sync is enabled, it applies the Git state to the cluster. If they're in sync, it does nothing. If auto-sync is disabled, it reports the drift for a human to address. The Application Controller runs in-cluster and uses the cluster's own service account to apply changes, which means cluster credentials never need to leave the cluster.

Dex is an embedded OpenID Connect identity provider. It federates authentication from external providers (GitHub, Google, LDAP, SAML) so ArgoCD users can log in with their existing organizational credentials rather than ArgoCD-specific accounts. Dex is optional but standard in enterprise setups.

Redis provides caching for the Application Controller. Rather than re-rendering the complete Git state on every reconciliation loop, ArgoCD caches rendered manifests in Redis. When the repository hasn't changed (most reconciliation cycles), the cached result is used. This dramatically reduces Git API calls and Repository Server load.

The Application object is ArgoCD's central custom resource. An Application tells ArgoCD: which Git repository to watch (and which path within it), which Kubernetes cluster and namespace to deploy to, which rendering tool to use (Helm, Kustomize, plain YAML), what sync policy to apply (manual or automatic), and what health checks to use.

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: model-server
  namespace: argocd
spec:
  project: ml-platform          # AppProject for RBAC scoping

  source:
    repoURL: https://github.com/myorg/gitops-config.git
    targetRevision: main        # Branch, tag, or commit SHA
    path: environments/production/model-server

    helm:                      # Helm-specific rendering config
      releaseName: model-server
      valueFiles:
        - values.yaml
        - values-production.yaml

  destination:
    server: https://kubernetes.default.svc # In-cluster deployment
    namespace: ml-production

  syncPolicy:
    automated:
      prune: true              # Delete resources removed from Git
      selfHeal: true           # Auto-revert manual changes
      allowEmpty: false        # Never sync empty app (safety check)
    syncOptions:
      - CreateNamespace=true
      - PrunePropagationPolicy=foreground
    retry:
      limit: 5
      backoff:
        duration: 5s
        factor: 2
        maxDuration: 3m

```

AppProject scopes what a group of Applications can deploy to. An AppProject can restrict which source repositories are allowed, which destination clusters and namespaces are allowed, which Kubernetes resource types can be deployed, and which teams can manage applications in the project. This is the multi-tenancy mechanism — the ML platform team's AppProject allows deploying to ML namespaces from ML repositories, while the data engineering team's AppProject scopes them to data namespaces from data repositories. Neither can accidentally (or intentionally) deploy to the other's territory.

04 Helm Chart Fundamentals

Helm is the package manager for Kubernetes. Just as `pip install` installs a Python package with all its dependencies configured correctly, `helm install` deploys a Kubernetes application with all its resources configured correctly for your specific needs.

The motivation for Helm is that real Kubernetes applications are not a single YAML file. A complete deployment typically needs: a Deployment, a Service, a ConfigMap, a Secret, an Ingress, an HPA, ServiceMonitor for Prometheus, RBAC roles and bindings, and possibly a PersistentVolumeClaim. That's 8+ separate YAML files, all needing consistent naming, labels, and configuration. Copying and modifying these files for each application or each environment is tedious, error-prone, and produces unmaintainable duplicated YAML.

Helm introduces templating — you write YAML with placeholders, and Helm fills in the values when installing. The template files, their default values, and metadata are packaged together into a Chart.

Chart structure:

diagram

12 lines

```
my-model-server/
├── Chart.yaml           # Chart metadata (name, version, description)
├── values.yaml          # Default configuration values
├── charts/              # Dependencies (sub-charts)
└── templates/           # Kubernetes manifest templates
    ├── deployment.yaml
    ├── service.yaml
    ├── ingress.yaml
    ├── hpa.yaml
    ├── servicemonitor.yaml
    └── _helpers.tpl     # Named template snippets (not rendered directly)
```

Chart.yaml defines the chart's identity:

yaml

12 lines

```
apiVersion: v2
name: model-server
description: ML model serving infrastructure
type: application
version: 1.3.0           # Chart version (changes when chart structure changes)
appVersion: "2.1.0"      # Application version being deployed
dependencies:
- name: redis
  version: "17.x.x"
  repository: https://charts.bitnami.com/bitnami
  condition: redis.enabled
```

values.yaml contains all configurable defaults:

```
replicaCount: 3

image:
  repository: myregistry/model-server
  tag: "latest"
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80
  targetPort: 8080

ingress:
  enabled: false
  host: ""
  tls: []

resources:
  requests:
    memory: "4Gi"
    cpu: "1"
  limits:
    memory: "8Gi"
    cpu: "2"

autoscaling:
  enabled: false
  minReplicas: 2
  maxReplicas: 10
  targetCPUUtilizationPercentage: 70

modelConfig:
  version: "v2.1.0"
  registryUrl: "s3://models/fraud-detection"
  batchSize: 32

monitoring:
  enabled: true
  port: 9090
```

A template file uses Go templating to reference values:

```

# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "model-server.fullname" . }}
  labels:
    {{- include "model-server.labels" . | nindent 4 }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "model-server.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "model-server.selectorLabels" . | nindent 8 }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          env:
            - name: MODEL_VERSION
              value: {{ .Values.modelConfig.version | quote }}
            - name: BATCH_SIZE
              value: {{ .Values.modelConfig.batchSize | quote }}
          resources:
            {{- toYaml .Values.resources | nindent 10 }}
            {{- if .Values.monitoring.enabled }}
          ports:
            - name: metrics
              containerPort: {{ .Values.monitoring.port }}
            {{- end }}

```

Helm release lifecycle:

bash

27 lines

```

# Install with default values
helm install my-model-server ./model-server

# Install with custom values
helm install my-model-server ./model-server \
  --values values-production.yaml \

```

```

--set image.tag=v2.1.0 \
--set replicaCount=5 \
--namespace ml-production

# Upgrade an existing release
helm upgrade my-model-server ./model-server \
  --values values-production.yaml \
  --set image.tag=v2.2.0

# Preview what would be deployed without actually deploying
helm template my-model-server ./model-server --values values-production.yaml

# Rollback to previous release version
helm rollback my-model-server 1

# Uninstall all resources for a release
helm uninstall my-model-server

# View deployed releases
helm list --namespace ml-production

```

Helm maintains release history — every install and upgrade is recorded. This history enables rollback and provides an audit trail within the Helm ecosystem, though for GitOps the Git history is the authoritative record.

05 Helm Templating & Values Management

Helm templating is Go's template language applied to YAML, and mastering it is what separates functional Helm charts from excellent ones. The values management strategy — how you organize and override values across environments — determines how maintainable your Helm-based GitOps configuration is.

Go template basics for Helm:

`{{ .Values.key }}` — reference a value from values.yaml. Nested values use dot notation: `{{ .Values.image.repository }}`

`{{ .Release.Name }}` — the name given at helm install time. Used extensively for naming resources.

`{{ .Chart.Name }}` and `{{ .Chart.Version }}` — metadata from Chart.yaml.

`{{- ... -}}` — the `-` trims whitespace before or after the action, preventing blank lines in output.

`{{ if .Values.feature.enabled }}...{{ end }}` — conditional rendering. Resources or blocks can be conditionally included.

`{{ range .Values.list }}...{{ end }}` — iterate over a list or map.

`{{ toYaml .Values.resources | nindent 10 }}` — convert a values map to YAML and indent it. Essential for embedding complex structures like resource requests.

`{{ required "Image tag is required" .Values.image.tag }}` — fail template rendering with a message if a value is not provided.

`{{ default "latest" .Values.image.tag }}` — use a fallback value if the specified value is empty.

Named templates in `_helpers.tpl` avoid repetition across template files:

yaml

16 lines

```
# templates/_helpers.tpl

{{/* Full name for resources */}}
{{- define "model-server.fullname" -}}
{{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" }}
{{- end }}

{{/* Standard labels applied to all resources */}}
{{- define "model-server.labels" -}}
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}
app.kubernetes.io/name: {{ .Chart.Name }}
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}
```

These named templates are called with `{{ include "model-server.fullname" . }}` throughout your templates, ensuring consistent naming without repetition.

Values management across environments is where many teams struggle. The wrong approach is having separate complete values files for each environment, duplicating 95% of content. The right approach is layering — a base values.yaml with sensible defaults, and per-environment override files with only what differs.

yaml

34 lines

```
# values.yaml (base defaults - committed to chart)
replicaCount: 2
image:
  tag: "latest"
resources:
  requests:
    memory: "2Gi"
    cpu: "0.5"

# values-staging.yaml (staging overrides only)
replicaCount: 2
image:
  tag: "" # Must be provided at deploy time
resources:
```

```

requests:
  memory: "4Gi"
  cpu: "1"

# values-production.yaml (production overrides only)
replicaCount: 10
image:
  tag: "" # Must be provided at deploy time
resources:
  requests:
    memory: "8Gi"
    cpu: "2"
  limits:
    memory: "16Gi"
    cpu: "4"
autoscaling:
  enabled: true
  minReplicas: 5
  maxReplicas: 50

```

With this structure, deploying to production means merging: base values.yaml (provides everything not overridden) + values-production.yaml (overrides environment-specific settings) + deploy-time `--set` flags (provides dynamic values like the specific image tag from the CI pipeline).

Schema validation for Helm values using `values.schema.json` prevents misconfiguration:

json

24 lines

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "required": ["image"],
  "properties": {
    "image": {
      "type": "object",
      "required": ["repository", "tag"],
      "properties": {
        "repository": {"type": "string"},
        "tag": {
          "type": "string",
          "pattern": "^v[0-9]+\\.?[0-9]+\\.?[0-9]+$"
        }
      }
    },
    "replicaCount": {
      "type": "integer",
      "minimum": 1,
      "maximum": 100
    }
  }
}

```

```
}  
}  
}
```

If a values file provides `image.tag: "latest"` instead of a proper version tag, Helm validates against this schema and rejects the deployment before it reaches the cluster.

Helm hooks execute jobs at specific points in the release lifecycle. Pre-install hooks run jobs before resources are created (database migrations, secret provisioning). Post-install hooks run after resources are ready (smoke tests, notifications). Pre-upgrade and post-upgrade hooks handle the same for upgrades. A database migration hook ensures schema changes happen before the new application version starts:

yaml

18 lines

```
# templates/migrate-job.yaml  
apiVersion: batch/v1  
kind: Job  
metadata:  
  name: {{ include "app.fullname" . }}-migrate  
  annotations:  
    "helm.sh/hook": pre-upgrade  
    "helm.sh/hook-weight": "-5"  
    "helm.sh/hook-delete-policy": hook-succeeded  
spec:  
  template:  
    spec:  
      containers:  
        - name: migrate  
          image: {{ .Values.image.repository }}:{{ .Values.image.tag }}  
          command: ["python", "manage.py", "migrate"]  
          restartPolicy: Never
```

06 Kustomize Basics

Kustomize is an alternative to Helm for Kubernetes manifest management. Where Helm uses templating (inserting values into placeholders in template files), Kustomize uses overlays (starting with base YAML files and applying structured patches on top). Neither approach is universally better — they have different strengths and are appropriate for different use cases.

The Kustomize philosophy: base Kubernetes YAML files should be valid, complete YAML that can be applied directly. Customizations for different environments are applied as separate patches, not by inserting variables into templates. This means base configurations are human-readable and runnable without Kustomize, while templated Helm charts are unreadable without rendering.

Kustomize structure:

diagram

21 lines

```
app/
├── base/                                # Valid, complete base manifests
│   ├── kustomization.yaml
│   ├── deployment.yaml
│   ├── service.yaml
│   └── configmap.yaml
└── overlays/
    ├── development/
    │   ├── kustomization.yaml
    │   └── patches/
    │       └── replica-count.yaml
    ├── staging/
    │   ├── kustomization.yaml
    │   ├── patches/
    │   └── resources.yaml
    └── production/
        ├── kustomization.yaml
        ├── patches/
        │   ├── replicas-and-resources.yaml
        │   └── hpa.yaml
```

Base kustomization.yaml lists the resources it includes:

yaml

17 lines

```
# base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- deployment.yaml
- service.yaml
- configmap.yaml

commonLabels:
  app: model-server
  managed-by: kustomize

images:
- name: myregistry/model-server
  newTag: latest
```

Overlay kustomization.yaml references the base and applies changes:

yaml

21 lines

```
# overlays/production/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

bases:
- ../../base

namespace: ml-production

namePrefix: prod-

images:
- name: myregistry/model-server
  newTag: v2.1.0           # Override the image tag

patches:
- path: patches/replicas-and-resources.yaml
  target:
    kind: Deployment
    name: model-server
```

Patch file specifies what to change:

yaml

19 lines

```
# overlays/production/patches/replicas-and-resources.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: model-server
spec:
  replicas: 10
  template:
    spec:
      containers:
      - name: model-server
        resources:
          requests:
            memory: "8Gi"
            cpu: "2"
          limits:
            memory: "16Gi"
            cpu: "4"
```

Kustomize applies this patch to the base Deployment, changing only the specified fields. Everything else from the base Deployment remains unchanged.

Strategic merge patches (like the above) understand Kubernetes semantics — they merge lists intelligently. A patch adding a container to a pod's container list adds it alongside existing containers, rather than replacing all containers with just the patched one. This is more intelligent than a simple text merge.

JSON patches offer surgical precision for specific field modifications:

yaml

14 lines

```
patches:
- target:
    kind: Deployment
    name: model-server
  patch: |-
    - op: replace
      path: /spec/replicas
      value: 10
    - op: add
      path: /spec/template/spec/containers/0/env/-
      value:
        name: ENVIRONMENT
        value: production
```

When to use Kustomize vs Helm — Kustomize is excellent when: you want valid base YAML that can be applied directly, your customizations are structural changes to existing resources (adding labels, changing replicas, overriding images), you prefer composition over templating, or you're working primarily with Kubernetes-native resources. Helm is better when: you're packaging applications for others to install (Helm's distribution model via chart repositories is much better than Kustomize's), you need conditional logic and loops in your configuration, or you're installing complex third-party software where Helm charts are already maintained by the project.

Many teams use both — Helm for third-party applications (install cert-manager, ingress-nginx, Prometheus using their official Helm charts) and Kustomize for their own application deployments (cleaner, no templating complexity, valid base YAML).

07 Application Deployment via GitOps

GitOps-based application deployment is the practical workflow that ties together Git, ArgoCD, Helm/Kustomize, and your CI/CD pipeline into a complete delivery system. Understanding this end-to-end workflow — from a developer pushing code to a change running in production — is what makes GitOps concrete and operational rather than theoretical.

The workflow has two distinct phases: the CI phase (producing a validated artifact) and the CD phase (deploying that artifact declaratively).

The CI phase (traditional push-based pipeline): 1. Developer pushes code to a feature branch 2. CI pipeline triggers: build, test, lint, security scan 3. If all checks pass, Docker image is built and pushed to registry with a unique tag (commit SHA or semantic version) 4. The pipeline does NOT deploy directly to Kubernetes 5. Instead, it updates the GitOps repository with the new image tag

The GitOps repository update is the critical handoff step. The CI pipeline commits a change to the GitOps repository that changes the image tag from the old version to the new one:

bash

26 lines

```
# In CI pipeline, after pushing new image
NEW_TAG="v2.1.0-${GITHUB_SHA:0:7}"

# Clone the GitOps repository
git clone https://github.com/myorg/gitops-config.git
cd gitops-config

# Update the image tag (using yq for YAML editing)
yq eval ".image.tag = \"${NEW_TAG}\"" -i \
    environments/staging/model-server/values.yaml

# Or update a Kustomize image reference
cd environments/staging/model-server
kustomize edit set image myregistry/model-server:${NEW_TAG}

# Commit and push
git config user.email "ci@company.com"
git config user.name "CI Bot"
git add -A
git commit -m "chore: update model-server to ${NEW_TAG}"

Automated deployment from CI pipeline.
Source commit: ${GITHUB_SHA}
Build: ${GITHUB_RUN_ID}"
git push
```

The CD phase (pull-based GitOps): 1. ArgoCD watches the GitOps repository and detects the commit 2. ArgoCD renders the Helm chart or Kustomize overlay with the new values 3. ArgoCD compares rendered output against the cluster's current state 4. ArgoCD detects the image tag difference and applies the change 5. Kubernetes rolls out the new deployment according to its update strategy 6. ArgoCD monitors the rollout and reports health status

The ArgoCD Application watching for these changes:

yaml

24 lines

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: model-server-staging
  namespace: argocd
spec:
  project: ml-platform
```

```
source:
  repoURL: https://github.com/myorg/gitops-config.git
  targetRevision: main
  path: environments/staging/model-server
  helm:
    valueFiles:
      - values.yaml
destination:
  server: https://kubernetes.default.svc
  namespace: ml-staging
syncPolicy:
  automated:
    prune: true
    selfHeal: true
  syncOptions:
    - CreateNamespace=true
```

Separating application and GitOps repositories is a critical design decision. Some teams put Helm charts and GitOps configuration in the same repository as application code. This creates coupling — every code change triggers potential GitOps changes, and the repository history mixes application and deployment concerns. Better practice:

Application repository — application code, Dockerfile, CI pipeline. Produces Docker images.

GitOps repository (sometimes called config repo or manifests repo) — Helm charts, Kustomize overlays, environment-specific values. Consumed by ArgoCD. Only deployment-related changes. The CI pipeline from the application repo commits here to trigger deployments.

This separation means: reading the GitOps repository gives you a complete picture of what's deployed where. Changing deployment configuration doesn't require going through application CI. Access control can be different — more engineers have write access to the application repo, fewer (or automated systems only) update deployment tags.

08 Automated Sync & Self-Healing

Automated sync and self-healing are what transform ArgoCD from a deployment tool into a continuous reconciliation system. Without these features, ArgoCD is just a more auditable way to run `kubectl apply`. With them, it becomes a system that actively maintains declared state against the real world.

Automated sync means ArgoCD automatically applies Git changes to the cluster without requiring human action. When you push a commit to the GitOps repository, ArgoCD detects the change within a few minutes (based on polling interval, typically 3 minutes, or immediately via webhook) and automatically syncs the application.

Without automated sync, ArgoCD operates in manual mode — it detects drift and shows it in the UI, but a human must click "Sync" to apply it. Manual mode is appropriate for production environments where you want human oversight of every deployment. Automated sync is appropriate for development and staging environments where fast iteration matters more than manual review.

Configuring automated sync with important safeguards:

```
syncPolicy:
  automated:
    prune: true          # Delete resources that are in cluster but not in Git
    selfHeal: true       # Revert manual changes made outside Git
    allowEmpty: false    # Never sync an application that renders to zero resources
                        # (prevents accidentally deleting everything if rendering fails)
  syncOptions:
    - PrunePropagationPolicy=foreground # Wait for dependent resources to be deleted
    - RespectIgnoreDifferences=true     # Respect configured ignore rules
  retry:
    limit: 5              # Retry failed syncs up to 5 times
    backoff:
      duration: 5s
      factor: 2
      maxDuration: 3m     # Maximum wait between retries: 3 minutes
```

Prune is a critical setting that requires careful consideration. When you remove a resource from your GitOps repository (delete a Deployment YAML, remove a Service from your Helm chart), what should happen to the resource that already exists in the cluster? Without `prune: true`, the resource stays running indefinitely — ghost resources accumulate in your cluster that are no longer managed but continue consuming resources. With `prune: true`, ArgoCD deletes resources that exist in the cluster but are no longer declared in Git. This is the correct behavior for production but can be dangerous if misconfigured — a rendering error that produces empty output could delete all your resources if `allowEmpty: false` isn't set.

Self-healing is what makes GitOps's immutability property real. Without self-healing, someone with `kubectl` access can change a resource in the cluster, and it stays changed until the next explicit sync. With self-healing, ArgoCD detects any drift from the Git-declared state and automatically reverts it, typically within a few minutes.

This is one of GitOps's most powerful security properties. An attacker who gains `kubectl` access to your production cluster and modifies a deployment to run their code will find their changes reverted within the next reconciliation cycle. This doesn't make cluster access less important to protect, but it dramatically reduces the window of impact from unauthorized changes.

Webhook configuration for immediate sync instead of polling:

Rather than ArgoCD polling Git every 3 minutes, configure your Git provider to send a webhook notification to ArgoCD when code is pushed. ArgoCD receives the webhook, immediately fetches the latest state, and triggers a sync. This reduces the time from git push to deployment start from 3 minutes to seconds.

In GitHub, add a webhook at <https://argocd-server/api/webhook> with a shared secret. ArgoCD validates the webhook signature and triggers refresh immediately on receiving it.

09 GitOps Repository Structure Design

The structure of your GitOps repository is an architectural decision that affects how easily you can understand the current state of your systems, how quickly you can deploy to specific environments, how you manage permissions for different teams, and how you scale to many applications and environments. Getting

this right early prevents painful restructuring later.

There is no single universally correct structure, but there are well-established patterns with known tradeoffs.

Pattern 1: Environment-based top-level directories

diagram

26 lines

```
gitops-config/
├── environments/
│   ├── development/
│   │   ├── model-server/
│   │   │   ├── values.yaml
│   │   │   └── kustomization.yaml
│   │   └── feature-store/
│   │       └── values.yaml
│   ├── staging/
│   │   ├── model-server/
│   │   │   └── values.yaml
│   │   └── feature-store/
│   │       └── values.yaml
│   └── production/
│       ├── model-server/
│       │   └── values.yaml
│       └── feature-store/
│           └── values.yaml
├── charts/                                # Helm charts (if co-located)
│   ├── model-server/
│   └── feature-store/
└── apps/                                  # ArgoCD Application manifests
    ├── development.yaml                  # ArgoCD App of Apps
    ├── staging.yaml
    └── production.yaml
```

This structure makes it easy to answer "what's deployed in production?" — look in `environments/production/`. To deploy a new version, update values in the specific environment directory. CODEOWNERS can restrict who can modify `environments/production/` to senior engineers or automated systems.

Pattern 2: Application-based top-level directories

```
gitops-config/
├── model-server/
│   ├── base/
│   │   ├── deployment.yaml
│   │   └── service.yaml
│   └── overlays/
│       ├── development/
│       ├── staging/
│       └── production/
├── feature-store/
│   └── ...
└── argocd-apps/
    └── ...
```

This structure makes it easy to see all environments for a specific application in one place. Better for teams organized around services rather than environments.

App of Apps pattern — managing dozens of ArgoCD Application objects manually doesn't scale. The App of Apps pattern uses a parent Application that manages child Application objects, enabling you to define your entire application portfolio declaratively:

```
# apps/production.yaml - Parent Application
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: production-apps
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/myorg/gitops-config.git
    targetRevision: main
    path: argocd-apps/production    # Directory of Application manifests
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

```
argocd-apps/production/
├─ model-server.yaml      # ArgoCD Application for model-server
├─ feature-store.yaml     # ArgoCD Application for feature-store
├─ mlflow.yaml            # ArgoCD Application for MLflow
└─ monitoring.yaml        # ArgoCD Application for monitoring stack
```

When you add a new application, you add a new Application manifest to this directory. The parent App of Apps picks it up and creates the child Application automatically. Your entire production application portfolio is declaratively defined and version-controlled.

ApplicationSets extend this further with templating for Applications. Instead of writing 10 similar Application manifests for 10 environments or clusters, write one ApplicationSet template:

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: model-server
  namespace: argocd
spec:
  generators:
  - list:
      elements:
      - env: development
        cluster: dev-cluster
        url: https://dev-k8s.company.com
        namespace: ml-dev
        replicaCount: "1"
      - env: staging
        cluster: staging-cluster
        url: https://staging-k8s.company.com
        namespace: ml-staging
        replicaCount: "3"
      - env: production
        cluster: prod-cluster
        url: https://prod-k8s.company.com
        namespace: ml-production
        replicaCount: "10"
  template:
    metadata:
      name: 'model-server-{{env}}'
    spec:
      project: ml-platform
      source:
        repoURL: https://github.com/myorg/gitops-config.git
```

```

targetRevision: main
path: 'environments/{{env}}/model-server'
helm:
  parameters:
    - name: replicaCount
      value: '{{replicaCount}}'
destination:
  server: '{{url}}'
  namespace: '{{namespace}}'

```

One ApplicationSet generates three Applications. Adding a new environment means adding one entry to the generators list.

10 Environment Promotion Strategy

Promotion is the process of moving a change from one environment to the next — from development to staging to production. In GitOps, promotion means updating the GitOps repository's environment-specific configuration to reflect the new version. The strategy for how this happens determines your deployment velocity, safety, and auditability.

Manual promotion via pull request is the safest and most auditable approach for production changes. The process: automated or human creates a pull request that updates the production environment's image tag or configuration, designated reviewers (senior engineers, team leads) review the PR and the changes it represents, the PR is merged after approval, ArgoCD detects the merge and deploys automatically. Every production change has a PR record with reviewer names, timestamps, and discussion.

bash

23 lines

```

# Promotion script: promote staging to production
STAGING_VERSION=$(yq eval '.image.tag' environments/staging/model-server/values.yaml)

# Create a promotion branch
git checkout -b promote/model-server-`${STAGING_VERSION}`

# Update production values to match staging
yq eval ".image.tag = \"`${STAGING_VERSION}`\"" -i \
  environments/production/model-server/values.yaml

git add environments/production/model-server/values.yaml
git commit -m "promote: model-server `${STAGING_VERSION}` to production"

Promoting version that has been validated in staging.
Staging validation duration: 2 days
Key metrics: p99 latency 145ms, error rate 0.02%"

git push origin promote/model-server-`${STAGING_VERSION}`
# Create PR automatically via GitHub CLI
gh pr create \

```

```
--title "Promote model-server ${STAGING_VERSION} to production" \  
--body "Automated promotion. Review metrics dashboard before approving."
```

Automated promotion is used for development environments and can be used for staging with appropriate validation gates. After CI builds and tests pass, the pipeline automatically updates the next environment:

yaml

13 lines

```
# GitHub Actions: Auto-promote to staging after dev validation  
- name: Promote to staging  
  if: github.ref == 'refs/heads/main' && env.DEV_VALIDATION_PASSED == 'true'  
  run: |  
    cd gitops-config  
    NEW_TAG="${{ env.IMAGE_TAG }}"  
  
    yq eval ".image.tag = \"${NEW_TAG}\"" -i \  
      environments/staging/model-server/values.yaml  
  
    git commit -am "auto-promote: model-server ${NEW_TAG} to staging"  
    git push
```

Progressive promotion with validation gates — sophisticated promotion pipelines include automated validation before advancing to the next environment:

diagram

18 lines

```
CI passes → Deploy to dev (automated)  
↓  
Dev smoke tests pass (automated, ~5min)  
↓  
Deploy to staging (automated)  
↓  
Staging integration tests pass (automated, ~30min)  
↓  
Staging soak period (configured, 24-48 hours)  
↓  
Human approval (manual PR review)  
↓  
Deploy to production (automated after PR merge)  
↓  
Production canary metrics check (automated, 30min)  
↓  
Full production rollout (automated if metrics pass)
```

Each gate serves a purpose. Integration tests catch correctness issues. Soak periods catch memory leaks and gradual degradation that only appear over time. Human approval adds judgment for risk assessment. Canary metrics provide real-world validation on a subset of production traffic before full exposure.

Version tracking across environments — it's valuable to be able to see at a glance what version is in each environment. A simple approach is a dashboard script or a GitHub Actions job that reads each environment's `values.yaml` and reports:

bash

11 lines

```
#!/bin/bash
echo "≡≡≡ Environment Version Status ≡≡≡"
for env in development staging production; do
    VERSION=$(yq eval '.image.tag' environments/${env}/model-server/values.yaml)
    echo "${env}: ${VERSION}"
done
# Output:
# development: v2.2.0-abc1234
# staging: v2.1.0-def5678
# production: v2.0.0-ghi9012
```

11 Drift Detection & Reconciliation

Drift is the divergence between the desired state declared in Git and the actual state running in your cluster. It's an inevitability in any real system — manual interventions during incidents, partially applied changes, configuration changes made through cloud provider consoles, Kubernetes operators modifying resources, or simply bugs in the reconciliation process.

How drift occurs in practice:

On-call engineers making emergency changes — someone SSHes into a node or runs `kubectl edit` during an incident at 2am to fix a production problem. They forget to update the GitOps repository with their change. The fix stays in place, but Git doesn't know about it.

Kubernetes operators and controllers modifying resources — a cluster autoscaler updates node group sizes, a certificate controller updates TLS certificates, an HPA changes replica counts. These are legitimate changes by authorized systems, but they create differences between Git state and cluster state.

Partially applied changes — a sync that fails halfway through leaves the cluster in an intermediate state that matches neither the old Git state nor the new one.

Namespace or cluster-level configurations made outside GitOps — cluster administrators make cluster-wide changes (new node taints, cluster-level RBAC) outside the GitOps process.

ArgoCD's drift detection compares the live state of every managed resource against the desired state from Git. The comparison is performed by the Application Controller on each reconciliation cycle (default every 3 minutes, or triggered by webhooks). It uses a sophisticated diff algorithm that understands Kubernetes conventions — it ignores auto-generated fields like `creationTimestamp`, `generation`, `resourceVersion` that Kubernetes adds and that shouldn't be managed in Git.

Sync status in ArgoCD: - **Synced** — live state matches desired state completely - **OutOfSync** — differences detected between live and desired state - **Unknown** — ArgoCD can't determine the sync status (cluster unreachable, rendering failed)

Health status (separate from sync status): - **Healthy** — all resources report healthy (Deployment has desired replicas, all pods running) - **Progressing** — resources are still being created or updated - **Degraded** — something is wrong (pod crash loops, insufficient replicas) - **Missing** — resources declared in Git don't exist in the cluster

Configuring what to ignore — not all drift is undesirable. HPA changing replica counts is expected behavior, not unauthorized drift. ArgoCD's ignore differences configuration tells it which fields or resources to ignore during drift detection:

yaml

21 lines

```
spec:
  ignoreDifferences:
    # Ignore HPA-managed replicas
    - group: apps
      kind: Deployment
      jsonPointers:
        - /spec/replicas

    # Ignore auto-generated annotations
    - group: ""
      kind: Service
      jsonPointers:
        - /metadata/annotations/kubectl.kubernetes.io~1last-applied-configuration

    # Ignore cert-manager managed certificate data
    - group: ""
      kind: Secret
      name: tls-secret
      jsonPointers:
        - /data
```

Manual drift correction — when self-healing is disabled (appropriate for production with careful change management), the workflow for correcting drift is explicit. You detect the drift (ArgoCD shows OutOfSync), you investigate what changed and whether it was intentional, you either update the GitOps repository to match the manual change (if the manual change was correct and should persist) or you sync to revert the manual change to the Git state.

Recording emergency changes — when on-call engineers make emergency changes directly to the cluster, they should immediately (or as soon as the incident is resolved) either open a PR to update the GitOps repository to reflect the change, or document why the change was made and that it will be reverted. Some teams add labels to manually-modified resources during incidents ([incident/INC-1234](#)) as a reminder to reconcile after.

12 Secret Management in GitOps

Secret management in GitOps is one of the hardest problems in the entire GitOps workflow. The fundamental tension: GitOps requires that the complete desired state of your system is in Git, but secrets (database passwords, API keys, TLS certificates) must never be committed to Git in plain text — Git repositories can be accessed by many people and potentially leaked.

Several approaches address this, with different tradeoffs.

Sealed Secrets (Bitnami) encrypts secrets using a public key before committing them to Git. The corresponding private key lives only in the cluster. The SealedSecret controller decrypts SealedSecrets and creates regular Kubernetes Secrets. Only the cluster with the matching private key can decrypt them.

bash

15 lines

```
# Install sealed-secrets controller and kubeseal CLI
# Get the cluster's public key
kubeseal --fetch-cert > sealed-secrets-public-key.pem

# Create and encrypt a secret
kubectl create secret generic db-credentials \
  --from-literal=username=mluser \
  --from-literal=password=supersecret \
  --dry-run=client -o yaml | \
kubeseal --cert sealed-secrets-public-key.pem \
  --format yaml > sealed-secrets/db-credentials.yaml

# The sealed-secrets/db-credentials.yaml is safe to commit to Git
# It looks like:
```

yaml

15 lines

```
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: db-credentials
  namespace: ml-production
spec:
  encryptedData:
    username: AgBy3i... # Encrypted value
    password: AgCk4j... # Encrypted value
  template:
    metadata:
      name: db-credentials
      namespace: ml-production
    type: Opaque
```

The SealedSecret object can be committed to Git. ArgoCD deploys it to the cluster. The sealed-secrets controller decrypts it and creates a regular Kubernetes Secret. If someone reads the Git repository, they see the encrypted values — useless without the private key that only exists in the cluster.

The limitation: if the cluster's private key is lost, all sealed secrets are unrecoverable. Key backup and rotation requires resealing all secrets.

External Secrets Operator (ESO) is the most production-ready approach. Secrets live in an external secrets management system (AWS Secrets Manager, Vault, GCP Secret Manager, Azure Key Vault). The External Secrets Operator runs in Kubernetes, authenticates to the external secrets system, and creates Kubernetes Secrets from the external values. What gets committed to Git is the ExternalSecret object — a declarative reference to where the secret lives, not the secret value itself.

yaml

24 lines

```
# This is safe to commit to Git - contains no secret values
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-credentials
  namespace: ml-production
spec:
  refreshInterval: 1h                # Re-sync from source every hour
  secretStoreRef:
    name: aws-secretsmanager         # Reference to a SecretStore
    kind: ClusterSecretStore
  target:
    name: db-credentials             # Creates this Kubernetes Secret
    creationPolicy: Owner
  data:
    - secretKey: username            # Key in the Kubernetes Secret
      remoteRef:
        key: ml-production/db-credentials # Key in AWS Secrets Manager
        property: username
    - secretKey: password
      remoteRef:
        key: ml-production/db-credentials
        property: password
```

The SecretStore configures how to authenticate to AWS Secrets Manager (typically via IRSA — IAM Roles for Service Accounts):

```
apiVersion: external-secrets.io/v1beta1
kind: ClusterSecretStore
metadata:
  name: aws-secretsmanager
spec:
  provider:
    aws:
      service: SecretsManager
      region: us-east-1
      auth:
        jwt:
          serviceAccountRef:
            name: external-secrets-sa
            namespace: external-secrets
```

ESO's advantages over Sealed Secrets: secrets are managed in a purpose-built system with audit logging, access control, rotation, and versioning. Rotating a secret means updating it in Secrets Manager — ESO automatically picks up the new value within the refresh interval. No re-sealing required. Secret access is audited at the secrets manager level.

SOPS (Secrets OPERATIONs) encrypts specific values within YAML files while keeping the structure readable:

```
# Before encryption (not committed)
database:
  username: mluser
  password: supersecret

# After sops encryption (safe to commit)
database:
  username: ENC[AES256_GCM,data:abc123,tag:xyz]
  password: ENC[AES256_GCM,data:def456,tag:uvw]
sops:
  kms:
    - arn: arn:aws:kms:us-east-1:123456789:key/my-key
```

SOPS integrates with ArgoCD through the KSOPS or HELM-SECRETS plugins, which decrypt secrets during the rendering phase.

13 Rollback Strategies in GitOps

In GitOps, a rollback is just another git operation. Because Git is the source of truth and every state the system has ever been in is preserved in Git history, rolling back means pointing the system back to a previous Git state. This conceptual simplicity is one of GitOps's major operational advantages.

Git revert creates a new commit that undoes the changes of a previous commit, adding to history rather than rewriting it. This is the preferred production rollback method because it preserves audit trail — the deployment was made, it was found to be problematic, it was reverted, all events recorded:

bash

14 lines

```
# Identify the commit that caused the problem
git log --oneline environments/production/model-server/

# a1b2c3d deploy: model-server v2.2.0 to production
# d4e5f6g deploy: model-server v2.1.0 to production
# g7h8i9j deploy: model-server v2.0.0 to production

# Revert the problematic deployment
git revert a1b2c3d --no-edit
git push origin main

# ArgoCD detects the new commit (which reverts to v2.1.0 image tag)
# and syncs the cluster back to v2.1.0
```

The revert commit in Git clearly shows: "model-server v2.2.0 was reverted because [reason from commit message]." This is valuable for post-incident reviews and compliance.

Direct image tag update for faster emergency rollback when speed matters more than pristine history:

bash

15 lines

```
# Quickly update the image tag to the previous known-good version
PREVIOUS_VERSION="v2.1.0"

yq eval ".image.tag = \"${PREVIOUS_VERSION}\"" -i \
    environments/production/model-server/values.yaml

git add environments/production/model-server/values.yaml
git commit -m "emergency rollback: model-server to ${PREVIOUS_VERSION}"

INCIDENT: INC-1234
Reason: v2.2.0 causing elevated error rates on transaction processing
Rolling back to last known good version
"
git push
```

ArgoCD sync to previous revision — ArgoCD can sync to a specific Git commit without you needing to do a git revert or push:

bash

6 lines

```
# Rollback to a specific revision using ArgoCD CLI
argocd app rollback model-server-production \
    --revision d4e5f6g      # The commit SHA to roll back to

# Or in the ArgoCD UI: History and Rollback → select revision → Rollback
```

This is the fastest rollback method — no Git operations required — but it creates a drift state: the cluster is running what was in Git at commit d4e5f6g, but the current HEAD of Git has different content. ArgoCD shows OutOfSync against HEAD. You should follow up with a proper Git revert to bring Git back in line with what's running.

Automated rollback with sync waves and health checks:

ArgoCD's sync waves and health assessment enable automated rollback when a deployment doesn't become healthy. Configure sync wave ordering and rely on ArgoCD's health assessment:

yaml

6 lines

```
# Canary deployment first, main deployment second
# If canary isn't healthy, sync stops before touching main
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "-1"  # Deploy first (lower = earlier)
```

Combined with Argo Rollouts for progressive delivery, automated rollback happens when metrics violate thresholds during a canary deployment, without any human intervention.

Preventing rollback foot-guns — some rollback scenarios require care. Rolling back to a version that used a different database schema requires ensuring the database is also compatible. Rolling back a model version in production while keeping the new feature store schema could cause serving errors if the old model doesn't understand the new feature format. GitOps rollback handles the Kubernetes state — it doesn't automatically handle dependent data systems. Document data dependencies in your promotion and rollback runbooks.

14 Multi-Cluster Deployment Patterns

Most production ML systems eventually span multiple Kubernetes clusters — separate clusters for different environments, different regions, different cloud providers, or different tenants. GitOps with multiple clusters adds complexity but also enables powerful deployment patterns.

Why multiple clusters:

Environment isolation — truly separating production from staging requires separate clusters, not just separate namespaces. A node failure in a shared cluster could take down both environments simultaneously.

Geographic distribution — serving users in Asia, Europe, and North America with acceptable latency requires clusters in multiple regions. Model serving for real-time predictions can't tolerate cross-continental network latency.

Cloud provider diversity — avoiding vendor lock-in or meeting specific regulatory requirements may require clusters in different clouds.

Blast radius limitation — a critical bug in an operator or a cluster upgrade gone wrong only affects one cluster, not everything.

ArgoCD multi-cluster management:

ArgoCD can manage multiple clusters from a single ArgoCD installation (hub-and-spoke model) or from multiple ArgoCD installations (one per cluster or per environment).

In the hub-and-spoke model, one ArgoCD instance in a management cluster has credentials for all target clusters:

yaml

29 lines

```
# ApplicationSet deploying to multiple clusters
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: model-server-global
  namespace: argocd
spec:
  generators:
  - clusters:
      selector:
        matchLabels:
          environment: production # Select all clusters labeled as production
  template:
    metadata:
      name: 'model-server-{{name}}'
    spec:
      project: ml-platform
      source:
        repoURL: https://github.com/myorg/gitops-config.git
        targetRevision: main
        path: 'clusters/{{name}}/model-server'
      destination:
        server: '{{server}}'
        namespace: ml-production
      syncPolicy:
        automated:
          selfHeal: true
          prune: true
```

This ApplicationSet automatically creates Applications for every cluster with the `environment: production` label. When you add a new region cluster and label it appropriately, ArgoCD automatically starts managing it.

GitOps repository structure for multi-cluster:

```

gitops-config/
├── base/                                # Shared base configurations
│   ├── model-server/
│   │   ├── deployment.yaml
│   │   └── service.yaml
│   └── clusters/
│       ├── us-east-1/
│       │   ├── cluster.yaml            # Cluster metadata/labels
│       │   ├── model-server/
│       │   │   └── values.yaml         # US-East specific values
│       │   └── eu-west-1/
│       │       ├── model-server/
│       │       │   └── values.yaml     # EU-West specific values
│       │       └── ap-southeast-1/
│       │           ├── model-server/
│       │           │   └── values.yaml
│       └── environments/
│           ├── staging/
│           └── production/
├── argocd-apps/
│   ├── applicationsets/
│   │   └── model-server.yaml          # ApplicationSet for all clusters
│   └── apps/

```

Progressive multi-cluster rollout — when deploying a new model version globally, you don't want to update all regions simultaneously. A failure in one region during rollout should halt the rollout to remaining regions:

```

Deploy to us-east-1 (primary region, ~10% of traffic)
  ↓ Wait 30 minutes, validate metrics
Deploy to eu-west-1 (25% of traffic)
  ↓ Wait 30 minutes, validate metrics
Deploy to ap-southeast-1 (remaining traffic)
  ↓ Validate global metrics
Complete

```

This requires tooling beyond standard ArgoCD — either custom deployment pipelines that update cluster-specific values files progressively, or Argo Rollouts with analysis templates that gate progression based on per-cluster metrics.

Cluster registration in ArgoCD:

```
# Add a new cluster to ArgoCD management
argocd cluster add my-new-cluster-context \
  --name production-us-west-2 \
  --label environment=production \
  --label region=us-west-2

# ArgoCD creates a service account in the target cluster
# with necessary RBAC permissions and stores the credentials
```

15 Observability in GitOps Workflows

Observability in GitOps means being able to answer questions about your deployment process itself — not just whether your applications are healthy, but whether your GitOps system is working correctly, what changes are being deployed, how long deployments take, and what's causing sync failures.

ArgoCD metrics — ArgoCD exposes Prometheus metrics that give deep insight into the GitOps system's operation:

```
# Number of applications in each sync status
argocd_app_info{sync_status="OutOfSync"} # Applications that have drifted

# Application health status
argocd_app_info{health_status="Degraded"} # Applications with health problems

# Sync operation duration - how long do deployments take?
histogram_quantile(0.99,
  rate(argocd_app_sync_duration_seconds_bucket[1h])
)

# Repository reconciliation time
rate(argocd_repo_pending_request_total[5m])

# Number of sync operations over time (deployment frequency metric)
rate(argocd_app_sync_total[1h])
```

DORA metrics through GitOps — the four DORA (DevOps Research and Assessment) metrics measure software delivery performance:

Deployment frequency is the rate of `argocd_app_sync_total` for production applications.

Lead time for changes is the time from commit to the GitOps repository until that commit is reflected in the cluster. Calculate by recording the commit timestamp in the deployment commit and comparing to the ArgoCD sync completion time.

Change failure rate is the percentage of deployments that resulted in rollbacks or incidents. Track revert commits in the GitOps repository and incidents linked to deployments.

Time to restore service is measured from the first failed sync or health degradation event to restoration.

Deployment audit trail — every change to the GitOps repository creates a Git commit with author, timestamp, and message. Every ArgoCD sync operation is recorded in ArgoCD's operation history. Together, these create a complete audit trail: who committed what change when, and when that change was applied to which cluster.

For compliance requirements (SOX, HIPAA, PCI-DSS), this audit trail is valuable. ArgoCD's operation logs, combined with Git commit history, provide tamper-evident records of every production change.

Alerting on GitOps health:

yaml

32 lines

```
# Alertmanager rule for sync failures
groups:
- name: argocd-alerts
  rules:
  - alert: ArgoCDAppOutOfSync
    expr: |
      argocd_app_info{sync_status="OutOfSync", project="ml-platform"} == 1
    for: 30m
    labels:
      severity: warning
    annotations:
      summary: "ArgoCD application {{ $labels.name }} is out of sync"
      description: "Application has been OutOfSync for 30 minutes - manual intervention may be required"

  - alert: ArgoCDAppDegraded
    expr: |
      argocd_app_info{health_status="Degraded"} == 1
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "ArgoCD application {{ $labels.name }} is degraded"

  - alert: ArgoCDSyncFailed
    expr: |
      argocd_app_sync_total{phase="Error"} > 0
    for: 0m
    labels:
      severity: high
    annotations:
      summary: "ArgoCD sync failed for {{ $labels.name }}"
```

GitOps dashboard in Grafana — a dedicated dashboard for the GitOps layer shows: number of applications per health/sync status, deployment frequency over time, sync duration trends, recent sync operations and their outcomes, applications most frequently going OutOfSync (which might indicate unauthorized manual changes), and deployment timeline (which applications were deployed when, correlated with application metrics).

Notification integration — ArgoCD's notification controller sends deployment events to Slack, PagerDuty, or email:

yaml

22 lines

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-notifications-cm
data:
  trigger.on-sync-succeeded: |
    - description: Notify on successful sync
      send: [slack-deployment]
      when: app.status.operationState.phase in ['Succeeded']

  trigger.on-sync-failed: |
    - description: Alert on sync failure
      send: [slack-alerts, pagerduty]
      when: app.status.operationState.phase in ['Error', 'Failed']

  template.slack-deployment: |
    message: |
      :white_check_mark: *Deployed {{ .app.metadata.name }}*
      Version: {{ .app.status.summary.images | join ", " }}
      Commit: {{ .app.status.operationState.syncResult.revision | truncate 8 "" }}
      Author: {{ (call .repo.GetCommitMetadata .app.status.operationState.syncResult.revision).Author }}
```

Correlating deployment events with application metrics — in Grafana, add ArgoCD sync events as annotations on your application dashboards. When you see a latency spike at 14:23, you can immediately see that a deployment happened at 14:21, correlating the infrastructure event with the application impact. This is the GitOps-native equivalent of deployment markers on dashboards, and it works automatically because ArgoCD emits structured events that can be queried and annotated.

The thread connecting all of them is this: GitOps is fundamentally about making the deployment process auditable, reversible, and consistent by treating system state the same way good engineers treat code — versioned, reviewed, immutable once merged, and always recoverable. Every concept here, from ArgoCD's reconciliation loop to multi-cluster patterns to secret management, is a specific solution to a specific challenge in maintaining that discipline at production scale.

Internal Developer Platform (IDP)

01 Platform Engineering Fundamentals

To understand platform engineering, you first need to understand the problem that created the need for it, because the discipline emerged directly from a specific pain point that organizations hit as they scaled their engineering teams.

In the early days of DevOps, the solution to the problem of siloed development and operations teams was "you build it, you run it." Every engineering team became responsible for their own infrastructure, their own CI/CD pipelines, their own deployments, their own monitoring, their own security compliance. This was a huge improvement over the old model where developers threw code over a wall to operations and washed their hands of production concerns.

But as organizations scaled to dozens, hundreds, or thousands of engineers, a new problem emerged. Every team was solving the same infrastructure problems independently. Team A built a CI/CD pipeline for their service. Team B built a slightly different CI/CD pipeline for their service. Team C built yet another variation. Each team made different technology choices, different security configurations, different monitoring setups. The cumulative cognitive overhead was enormous — every engineer had to understand not just their domain problem but a huge surface area of infrastructure, cloud, Kubernetes, observability, security, and compliance tooling.

Worse, teams were spending significant engineering time on undifferentiated infrastructure work — work that didn't produce unique business value but was necessary to keep the lights on. A machine learning engineer spending 30% of their time figuring out how to configure Kubernetes resource limits and set up monitoring for their model serving service is a machine learning engineer spending 30% of their time on work that has nothing to do with machine learning.

Platform engineering is the discipline of building and operating internal platforms that enable other engineering teams to deliver software efficiently and reliably, while abstracting away the complexity of the underlying infrastructure. The platform team is a product team whose customers are other engineers. They build products — tools, workflows, self-service interfaces — that other teams use to do their work faster, with fewer errors, and without needing deep expertise in every infrastructure concern.

The mental model shift is critical: platform engineers are building a product, not providing a service. A service model means developers submit tickets and platform engineers fulfill requests. A product model means platform engineers build capabilities — golden paths, self-service portals, automated pipelines, standard templates — that developers use independently without involving the platform team for every individual request.

The core value proposition of platform engineering is leverage. One platform engineer building a capability that saves each of 100 development engineers 2 hours per week creates 200 engineer-hours of value per week from a single engineer's investment. The platform team's output is multiplied across the entire engineering organization.

For ML organizations specifically, this matters enormously. ML engineers, data scientists, and MLOps engineers have highly specialized skills. Time they spend wrestling with Kubernetes configurations, debugging CI/CD pipelines, or implementing monitoring from scratch is time not spent on model

development, feature engineering, and improving prediction quality. A platform that handles the infrastructure layer well — making it trivially easy to deploy a model serving endpoint, set up experiment tracking, or provision a training cluster — directly accelerates ML delivery.

02 DevOps vs Platform Engineering

DevOps and Platform Engineering are closely related but represent different stages in the evolution of how organizations manage the relationship between software development and infrastructure operations. Understanding the distinction helps you understand where platform engineering came from and what problem it solves that DevOps alone doesn't address.

DevOps is a culture and a set of practices that breaks down the silos between development and operations. The core idea is that the people who build software should also be responsible for running it in production. DevOps organizations eliminate the handoff between "developers write code" and "operations deploys and runs it" — instead, cross-functional teams own the full lifecycle from development through production.

DevOps brought enormous improvements: faster deployments, better reliability, more collaboration, less finger-pointing. But it assumed that development teams had the capability and capacity to manage their own infrastructure. It put the responsibility on developers, which meant developers needed to become proficient in infrastructure, cloud services, Kubernetes, monitoring, security, and compliance — a continuously growing set of concerns that had little to do with writing the actual software.

As organizations scaled, DevOps created a new problem: cognitive overload and fragmentation. When every team manages their own infrastructure independently, you get inconsistency, duplicated effort, and a massive aggregate investment in undifferentiated infrastructure work. The DevOps principle of teams owning their full stack is correct in spirit but doesn't scale efficiently when infrastructure complexity is high.

Platform Engineering is the organizational response to this scaling problem. Rather than every team independently solving the same infrastructure problems, a dedicated platform team solves them once and makes the solutions available to everyone through self-service products.

The key differences:

Scope of responsibility — DevOps teams own everything for their service: application code, CI/CD, infrastructure, monitoring, security. Platform engineers own the infrastructure products that other teams use. They don't own individual services; they own the platforms that enable service teams to operate independently.

Who the customer is — in DevOps, the customer is the end user of the software. In platform engineering, the customer is the internal developer. Platform engineers think constantly about developer experience, usability, and reducing friction for the engineers who use their platform.

How work is organized — DevOps is a cultural practice embedded in product teams. Platform engineering is a dedicated team (or multiple teams) with a product roadmap, customer interviews, adoption metrics, and an explicit mission to improve developer productivity.

What success looks like — DevOps success is measured by deployment frequency, lead time, MTTR, and change failure rate. Platform engineering success is measured by platform adoption, developer satisfaction, time-to-production for new services, reduction in undifferentiated infrastructure work, and the cognitive load reduction for product engineers.

The collaboration model — DevOps is about embedding operations thinking into development teams. Platform engineering is about extracting common infrastructure concerns from all development teams and solving them centrally, then giving those solutions back as self-service capabilities.

In practice, organizations need both. Platform engineering doesn't replace DevOps — it enables it at scale. Platform engineers build the golden paths and self-service tools. Development teams, practicing DevOps principles, use those tools to own their services end-to-end. The platform team handles the infrastructure complexity so development teams can focus on product complexity.

The distinction is also important for hiring and team design. Platform engineers need skills in infrastructure, developer experience, API design, automation, and product thinking. They're not just DevOps engineers doing the same work for many teams — they're building products that other engineers use as infrastructure.

03 Internal Developer Platform (IDP) Concepts

An Internal Developer Platform is the technical system that platform engineers build and operate — the concrete implementation of platform engineering principles. Where "platform engineering" describes the discipline and the team, "Internal Developer Platform" describes the product they build.

An IDP is a layer of abstraction between developers and the underlying infrastructure complexity. Developers interact with the IDP's self-service interface — APIs, UIs, CLI tools, templates — and the IDP translates those interactions into the necessary infrastructure operations: provisioning Kubernetes namespaces, setting up CI/CD pipelines, configuring monitoring, managing secrets, enforcing security policies.

The key properties that distinguish a real IDP from just a collection of tools:

Self-service — developers can provision what they need without waiting for a ticket to be fulfilled by the platform team. Need a new service deployment? Use the IDP's service template. Need a database? Use the IDP's database provisioning workflow. The entire interaction happens without human intervention from the platform team.

Opinionated — the IDP represents the platform team's considered opinions about the right way to build and operate software in this organization. It doesn't give developers unlimited flexibility — it gives developers carefully curated choices that encode best practices. This is intentional. Unlimited flexibility means every team makes different choices, which produces fragmentation. Opinionated platforms produce consistency.

Abstracted — developers interact with the IDP at a level of abstraction appropriate to their domain. An ML engineer should be able to deploy a model serving endpoint by specifying: what model to serve, what resource requirements it needs, and what environment it's for. They should not need to know about Kubernetes Deployments, Service objects, Ingress configurations, Prometheus ServiceMonitors, and HPA configurations. The IDP handles all of that from the high-level specification.

Composable — the IDP is built from components that can be combined. A service template composes a CI/CD pipeline template, a Kubernetes deployment template, a monitoring template, and a secrets management template. New service types are created by composing existing components in new ways.

Governed — the IDP enforces organizational policies automatically. Security requirements, compliance controls, resource limits, naming conventions — these are built into the IDP so that anything provisioned through it automatically complies. Governance becomes a property of the platform, not a gate that slows teams down.

The components that typically make up an IDP:

A developer portal (often Backstage) that provides the user-facing interface: service catalog, self-service templates, documentation, and operational visibility.

Service templates that produce complete, working services from simple specifications using scaffolding tools.

CI/CD automation that provides standard pipeline templates for building, testing, and deploying services.

Infrastructure automation that provisions cloud resources, Kubernetes configurations, monitoring, and security controls through self-service workflows.

An internal API layer that provides a unified interface to all these capabilities, enabling the portal, CLI tools, and programmatic access to work consistently.

A software catalog that tracks all the services, their ownership, their dependencies, their health, and their documentation — the organizational map of what exists.

04 Golden Path Strategy

The golden path is one of the most important concepts in platform engineering. It's the metaphor that best captures what a good internal platform provides and why it's valuable.

Imagine you're hiking in a national park. The park has marked trails — golden paths — that lead to the best views through safe terrain. You could go off-trail anywhere you want, but the marked trails exist because the park rangers have already found the best routes, marked them clearly, maintained them, and made sure they're safe to hike. Most visitors follow the marked trails, reach great destinations efficiently, and don't have to figure out route-finding from scratch. Expert hikers occasionally go off-trail for specific reasons, but even they might start on the marked trails.

In platform engineering, the golden path is the supported, documented, well-maintained path for common engineering tasks. It's the recommended way to build a new service, deploy a model, set up monitoring, manage configuration, or handle secrets. It's not the only way — teams with specialized needs can go off-path — but it's the path that the platform team has invested in making fast, safe, and well-supported.

The golden path strategy means the platform team explicitly identifies the most common things engineering teams need to do, designs the best possible workflow for doing each of those things, and then invests in making those workflows accessible, documented, and supported. The golden path is opinionated by design — it encodes the platform team's considered judgment about the right way to solve common problems.

What makes a golden path good:

It must be fast. If the golden path takes longer than rolling your own solution, nobody will use it. The golden path should be the fastest option for common tasks, not just the correct one.

It must be easy. The golden path should require minimal expertise in the underlying infrastructure. An ML engineer should be able to follow the golden path for deploying a model serving endpoint without knowing Kubernetes, without understanding networking, without knowing the monitoring stack. The complexity is hidden inside the path.

It must be complete. The golden path should produce a complete, production-ready result. A golden path that produces 80% of what you need and leaves teams to figure out the remaining 20% creates more friction than it removes. When you follow the golden path to create a new service, you should get a working service

with CI/CD, monitoring, logging, secrets management, and security controls — not just a Kubernetes Deployment that you then need to augment with ten other things.

It must be maintained. The golden path is infrastructure code that needs to be kept up to date. When Kubernetes upgrades, when security requirements change, when the organization adopts a new monitoring tool — the golden path should be updated. Teams using it inherit the improvements automatically. This is the compound interest of platform engineering: investment in the golden path benefits every team that uses it, forever.

It must be escapable. The golden path should be the default, not a prison. Teams with genuinely unusual requirements should be able to deviate from it. The platform team supports deviation graciously rather than fighting it — but they're honest that off-path choices mean less support and more self-reliance.

For an ML platform, golden paths might include: the path for training a new model (provisions compute, sets up experiment tracking, configures data access), the path for deploying a model to production (creates serving infrastructure, sets up monitoring, configures autoscaling, connects to the model registry), the path for creating a new data pipeline (provisions processing resources, sets up scheduling, configures observability), and the path for creating a new ML project (scaffolds repository structure, sets up CI/CD, creates experiment tracking project).

05 Self-Service Infrastructure

Self-service infrastructure is the operational model where engineers can provision the infrastructure they need on demand, without waiting for another team to fulfill a request. It's one of the most transformative capabilities a platform team can provide and one of the primary ways platforms accelerate organizational velocity.

The contrast with the old model is stark. In a traditional IT model, if you need a new database, you submit a ticket. The operations team reviews the request, asks clarifying questions, approves it, schedules the provisioning work, creates the database, configures it, and notifies you — a process that might take days or weeks. In a self-service model, you run a command or fill out a form, and within minutes your database is provisioned, configured according to organizational standards, and ready to use.

The business impact of this difference compounds over an entire engineering organization. Every time an engineer waits for infrastructure to be provisioned, they're blocked. Their work stalls, their context evaporates, they switch to something else and context-switch back later. Multiply that by hundreds of engineers making dozens of infrastructure requests per quarter, and the aggregate delay in velocity is enormous.

What makes self-service infrastructure possible is the combination of Infrastructure as Code (which makes provisioning programmatic and repeatable), cloud APIs (which enable on-demand resource creation), and platform tooling (which provides the user-friendly interface and policy enforcement layer on top of IaC and cloud APIs).

Self-service infrastructure doesn't mean unlimited self-service. The platform team designs the catalog of available infrastructure — which types of databases, which sizes, which cloud regions, which Kubernetes configurations — and engineers choose from that catalog. The platform team's policies are embedded in the provisioning system, so everything self-provisioned automatically meets security, compliance, and cost requirements.

The abstraction layer is key. Engineers shouldn't need to know how to write Terraform to provision infrastructure. They interact with a higher-level interface — a Backstage template, a CLI command, an API call — that translates their intent into the appropriate Terraform, Kubernetes manifests, or cloud API calls. The platform team maintains the IaC layer; engineers use the abstraction layer on top of it.

For ML teams, self-service infrastructure is particularly valuable because ML work requires a wide variety of infrastructure that changes throughout the development lifecycle. During exploration, a data scientist might need a Jupyter notebook environment with GPU access. During training, they need a training cluster. During evaluation, they need a model evaluation environment. During deployment, they need a model serving endpoint. Each of these traditionally would require separate tickets and waiting periods. Self-service means the data scientist provisions exactly what they need for each phase, uses it, and releases it when done — without any platform team involvement.

Cost management in self-service — giving teams unlimited self-service without guardrails is expensive. The platform needs to enforce quotas, implement automatic resource cleanup (destroying development environments after hours of inactivity), provide cost visibility (showing teams what their infrastructure costs), and implement approval workflows for large resource requests. Self-service doesn't mean uncontrolled — it means controlled efficiently.

06 Developer Experience (DevEx) Principles

Developer experience is how it feels to be an engineer at your organization. It's the sum of all the interactions, friction points, capabilities, and limitations that engineers encounter as they do their work. Good developer experience means engineers spend most of their time on work that's interesting, meaningful, and directly productive. Poor developer experience means engineers spend significant time fighting tools, waiting for systems, navigating bureaucracy, and doing work that feels wasteful.

Developer experience is the product that the platform team is building. Every platform decision — what to automate, what to leave manual, how to design APIs, what documentation to write, what error messages to display — affects developer experience.

The principles that guide good developer experience in platform engineering:

Optimize for the common case — identify the things engineers do most frequently and make those things as fast and friction-free as possible. Deploying a code change, checking the status of a deployment, debugging a failing service, reviewing logs — these happen dozens of times per day per engineer. Shaving 2 minutes off each of these saves enormous aggregate time. Rare administrative tasks can afford more friction.

Make the right thing easy and the wrong thing hard — good developer experience isn't just about speed. It's about guiding engineers toward correct behavior without requiring them to know all the rules. If the right way to handle secrets is through the secrets management system, make that the fastest and simplest option. If the wrong way is to hardcode secrets in environment variables, make that require extra steps and display warnings. Engineers following the path of least resistance should end up doing the right thing.

Fast feedback loops — when an engineer makes a change, they should know whether it worked as quickly as possible. A CI pipeline that takes 45 minutes before telling you about a typo is a developer experience failure. A deployment that shows you a dashboard of health metrics within 2 minutes of pushing code is developer experience done right. At every stage — local testing, CI, deployment, monitoring — the question is: how fast does the engineer know whether what they did worked?

Clear error messages — when something goes wrong, the error message should tell the engineer exactly what's wrong and how to fix it. "Error: validation failed" is a terrible error message. "Error: resource requests (cpu: 0) must be greater than zero. See docs.company.com/platform/resources for guidance on setting appropriate resource requests" is a great error message. The platform team should treat unclear error messages as bugs and fix them.

Progressive disclosure of complexity — the platform should be simple to use for common cases and allow engineers to access more complexity when they need it. A new engineer should be able to deploy their first service without understanding every configuration option. An expert engineer should be able to access deep configuration when their service has unusual requirements. Simple defaults cover most cases; advanced options are available but not required.

Documentation as a product — documentation is part of the developer experience, not an afterthought. Engineers should never need to ask the platform team "how do I do X?" — the documentation should answer common questions before they're asked. Platform teams should measure documentation quality by how often engineers still need to ask questions that should be answered in docs.

Measure what matters to engineers — developer satisfaction surveys, time-to-first-deployment for new hires, time spent on platform-related tasks, number of tickets submitted to the platform team, time waiting for infrastructure. These metrics tell you whether developer experience is improving or degrading over time.

07 Backstage Architecture Overview

Backstage is an open-source developer portal framework created by Spotify and donated to the CNCF. It's become the de facto standard for building internal developer portals — the central interface through which engineers interact with the internal developer platform.

Understanding Backstage's architecture is important because it determines what you can build with it, how to extend it for your organization's needs, and what operational responsibilities come with running it.

The core concept: the software catalog

At the heart of Backstage is a software catalog — a centralized registry of all software entities in your organization. Services, libraries, APIs, documentation sites, data pipelines, ML models, infrastructure components — anything meaningful can be registered in the catalog. Each catalog entry is a YAML file stored in the team's Git repository, describing the entity: its type, its owner, its dependencies, its links to documentation and dashboards.

yaml

32 lines

```
# catalog-info.yaml stored in service's repository
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: fraud-detection-model
  description: Real-time fraud detection model serving API
  tags:
    - ml
    - production
    - fraud-detection
```

```

annotations:
  github.com/project-slug: myorg/fraud-detection-model
  prometheus.io/alert: fraud_detection
  grafana/dashboard-selector: model-server
  backstage.io/techdocs-ref: dir:.
  argocd/app-name: fraud-detection-production
links:
  - url: https://grafana.company.com/d/fraud-model
    title: Monitoring Dashboard
  - url: https://mlflow.company.com/models/fraud-detection
    title: MLflow Registry
spec:
  type: ml-model
  lifecycle: production
  owner: group:ml-platform
  system: fraud-detection
  dependsOn:
    - component:feature-store
    - component:transaction-api
  providesApis:
    - fraud-detection-api

```

These catalog files are discovered by Backstage through repository scanning or explicit registration. The catalog aggregates them into a searchable, navigable directory of your entire software landscape.

Architecture components:

The **Frontend** is a React single-page application. It renders the developer portal UI — the catalog browser, service detail pages, templates, documentation, and any plugins you've installed. Backstage's plugin system allows extending the UI with new pages and functionality.

The **Backend** is a Node.js application that provides the API layer. It manages the catalog database, runs the catalog ingestion process (scanning repositories for catalog-info.yaml files), and provides APIs that the frontend consumes. The backend also provides authentication, authorization, and the plugin backend framework.

Plugins are the extension mechanism that makes Backstage valuable. The core Backstage installation provides a software catalog, basic search, and technical documentation. Everything else — GitHub integration, ArgoCD integration, Kubernetes deployment visibility, Prometheus metrics, cost monitoring, scaffolding templates, incident management, analytics — is a plugin. Backstage has a rich ecosystem of community plugins covering most common integrations, and the plugin API allows building custom plugins for organization-specific tools.

The **Software Templates** (Scaffolder) is a plugin that enables self-service creation of new services, repositories, and infrastructure. A template defines the steps to create something: render files from a template, create a GitHub repository, set up CI/CD, register in the catalog. Engineers fill out a form, and the scaffolder executes these steps automatically.

TechDocs is the documentation system. Backstage renders Markdown documentation stored in service repositories as browsable documentation pages, keeping documentation co-located with code but accessible through a central portal.

How Backstage pulls data together — the real power of Backstage is as an aggregation layer. Rather than engineers having to go to GitHub for code, ArgoCD for deployment status, Grafana for metrics, PagerDuty for incidents, MLflow for model status — Backstage pulls relevant information from all these systems and presents it in the context of a service. On the service page for your fraud detection model, you see the deployment status from ArgoCD, the current health metrics from Prometheus, the recent CI builds from GitHub Actions, the on-call rotation from PagerDuty, and the model version from MLflow — all without leaving Backstage.

08 Service Templates & Scaffolding

Service templates are the most impactful capability of a mature internal developer platform. A service template, used through Backstage's scaffolding system or equivalent tooling, takes a simple form input and automatically creates a complete, production-ready service with all the infrastructure correctly configured.

The value is enormous. Without templates, creating a new ML service might take an experienced engineer 1-2 days: create a repository, set up the project structure, configure the CI/CD pipeline, write Kubernetes manifests, set up monitoring, configure logging, create a model registry entry, register in the service catalog, add documentation. With a template, this takes 5 minutes and produces a more complete, correctly configured result than manual setup.

Anatomy of a Backstage Software Template:

A template is defined in a YAML file with three main sections: parameters (the form fields the engineer fills out), steps (the automated actions to execute), and output (links and information shown after completion).

yaml

142 lines

```
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: ml-model-serving-template
  title: ML Model Serving Service
  description: Create a new ML model serving service with complete infrastructure
  tags:
    - ml
    - model-serving
    - recommended
spec:
  owner: group:ml-platform
  type: service

  parameters:
    - title: Service Information
      required: [name, description, owner]
      properties:
        name:
          title: Service Name
          type: string
          description: Unique name for this model serving service
          pattern: '^[a-z][a-z0-9-]*[a-z0-9]$'
          ui:autofocus: true
```

```

    description:
      title: Description
      type: string
      description: Brief description of what this model serves
    owner:
      title: Owner Team
      type: string
      description: Team responsible for this service
      ui:field: OwnerPicker
      ui:options:
        allowedKinds: [Group]

- title: Model Configuration
  properties:
    modelType:
      title: Model Type
      type: string
      enum: [classification, regression, embedding, llm]
      enumNames: [Classification, Regression, Embedding, Large Language Model]
    gpuRequired:
      title: GPU Required
      type: boolean
      default: false
    framework:
      title: ML Framework
      type: string
      enum: [pytorch, tensorflow, sklearn, onnx]

- title: Infrastructure Configuration
  properties:
    environment:
      title: Initial Environment
      type: string
      enum: [development, staging, production]
      default: development
    replicaCount:
      title: Initial Replica Count
      type: integer
      minimum: 1
      maximum: 20
      default: 2
    memoryRequest:
      title: Memory Request
      type: string
      enum: [2Gi, 4Gi, 8Gi, 16Gi]
      default: 4Gi

steps:
- id: fetch-template
  name: Fetch Service Template
  action: fetch:template
  input:
    url: ./skeleton          # Template files directory
    values:
      name: ${ parameters.name }
      owner: ${ parameters.owner }
      modelType: ${ parameters.modelType }
      gpuRequired: ${ parameters.gpuRequired }
      framework: ${ parameters.framework }

- id: publish-github

```

```

name: Create GitHub Repository
action: publish:github
input:
  allowedHosts: ['github.com']
  description: ${ parameters.description }
  repoUrl: github.com?owner=myorg&repo=${ parameters.name }
  defaultBranch: main
  requireCodeOwner: true
  repoVisibility: private
  topics: ['ml', 'model-serving', ${ parameters.modelType }]

- id: create-argocd-app
  name: Register in ArgoCD
  action: argocd:create-application
  input:
    appName: ${ parameters.name }-${ parameters.environment }
    argoInstance: production
    projectName: ml-platform
    repoUrl: ${ steps.publish-github.output.remoteUrl }
    path: gitops/${ parameters.environment }
    namespace: ml-${ parameters.environment }

- id: trigger-ci
  name: Trigger Initial CI Pipeline
  action: github:actions:dispatch
  input:
    repoUrl: ${ steps.publish-github.output.remoteUrl }
    workflowId: ci.yml

- id: register-catalog
  name: Register in Service Catalog
  action: catalog:register
  input:
    repoContentsUrl: ${ steps.publish-github.output.repoContentsUrl }
    catalogInfoPath: /catalog-info.yaml

- id: create-mlflow-experiment
  name: Create MLflow Experiment
  action: http:backstage:request
  input:
    method: POST
    path: /api/mlflow/experiments
    body:
      name: ${ parameters.name }
      artifact_location: s3://mlflow-artifacts/${ parameters.name }

output:
  links:
    - title: View Repository
      url: ${ steps.publish-github.output.remoteUrl }
    - title: View in ArgoCD
      url: https://argocd.company.com/applications/${ parameters.name }-${ parameters.environment }
    - title: Service Dashboard
      url: https://backstage.company.com/catalog/default/component/${ parameters.name }
    - title: MLflow Experiment
      url: https://mlflow.company.com/experiments/${ parameters.name }

```

The **template skeleton** is the directory of file templates rendered with the engineer's input values:

```
skeleton/
├── .github/
│   └── workflows/
│       ├── ci.yml.jinja2
│       └── cd.yml.jinja2
├── src/
│   ├── server.py.jinja2
│   ├── predictor.py.jinja2
│   └── config.py.jinja2
├── gitops/
│   ├── development/
│   │   └── values.yaml.jinja2
│   └── production/
│       └── values.yaml.jinja2
├── Dockerfile.jinja2
├── requirements.txt.jinja2
├── catalog-info.yaml.jinja2
└── README.md.jinja2
```

Each file uses template variables like `{{ values.name }}`, `{{ values.gpuRequired }}` that are substituted with the engineer's form input during rendering.

The result: the engineer fills out a 3-page form, clicks Create, and 2 minutes later has a complete GitHub repository with a working FastAPI model server, CI/CD pipeline, GitOps configuration for deployment, monitoring setup, catalog entry, and MLflow experiment — all correctly named and configured, all complying with organizational standards, all ready for development to begin.

09 CI/CD Template Automation

CI/CD template automation solves the fragmentation problem where every team writes their own pipeline from scratch, producing hundreds of slightly different pipeline files that all need to be maintained, updated, and debugged independently.

The platform team defines standard CI/CD pipeline templates that implement organizational best practices — proper caching, security scanning, testing stages, artifact management, deployment patterns. Service teams adopt these templates rather than writing from scratch. When the platform team improves the template (adds a new security scan, improves caching, updates deployment logic), every team that uses the template benefits automatically.

Reusable GitHub Actions workflows are the most common implementation for GitHub-based organizations. The platform team maintains a `.github` repository with reusable workflow definitions:

```
# .github/workflows/ml-service-ci.yml in the platform .github repo
name: ML Service CI/CD
on:
```

workflow_call:

inputs:

service-name:

required: true

type: string

image-registry:

required: true

type: string

deploy-environment:

required: false

type: string

default: staging

secrets:

ecr-push-role:

required: true

gitops-token:

required: true

jobs:

test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Set up Python

uses: actions/setup-python@v4

with:

python-version: "3.11"

cache: pip

- name: Install dependencies

run: pip install -r requirements.txt -r requirements-dev.txt

- name: Run tests with coverage

run: |

pytest tests/ \

--junitxml=test-results.xml \

--cov=src \

--cov-report=xml \

--cov-fail-under=80

- name: Upload test results

uses: actions/upload-artifact@v3

with:

name: test-results

path: test-results.xml

security-scan:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Run bandit security scan

run: pip install bandit && bandit -r src/ -f json -o bandit-results.json

- name: Run dependency vulnerability scan

run: pip install safety && safety check --json

build-push:

```

needs: [test, security-scan]
runs-on: ubuntu-latest
permissions:
  id-token: write
  contents: read
outputs:
  image-digest: ${ steps.build.outputs.digest }
steps:
- uses: actions/checkout@v4

- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: ${ secrets.ecr-push-role }
    aws-region: us-east-1

- name: Build, scan, push
  id: build
  uses: myorg/.github/.github/actions/build-scan-push@v2
  with:
    image: ${ inputs.image-registry }/${ inputs.service-name }
    tag: ${ github.sha }

- name: Sign image
  run: |
    cosign sign --yes \
      ${ inputs.image-registry }/${ inputs.service-name }@${ steps.build.outputs.digest }

update-gitops:
needs: build-push
runs-on: ubuntu-latest
steps:
- name: Update GitOps repository
  uses: myorg/.github/.github/actions/update-gitops@v2
  with:
    service: ${ inputs.service-name }
    environment: ${ inputs.deploy-environment }
    image-tag: ${ github.sha }
    token: ${ secrets.gitops-token }

```

Service teams use this template in their own repository with a minimal workflow file:

```
# .github/workflows/ci.yml in the service repository
name: CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  pipeline:
    uses: myorg/.github/.github/workflows/ml-service-ci.yml@main
    with:
      service-name: fraud-detection-model
      image-registry: 123456789.dkr.ecr.us-east-1.amazonaws.com
      deploy-environment: staging
    secrets:
      ecr-push-role: ${ secrets.ECR_PUSH_ROLE }
      gitops-token: ${ secrets.GITOPS_TOKEN }
```

The service team's pipeline file is 15 lines. The platform team's reusable workflow is comprehensive and maintained centrally. When the platform team adds a new mandatory security scan, they update the reusable workflow and every service automatically gets the new scan on their next run.

Template versioning — reusable workflows should be versioned (using Git tags) so service teams can pin to a specific version:

```
uses: myorg/.github/.github/workflows/ml-service-ci.yml@v3
```

This allows the platform team to make breaking changes in v4 while services on v3 continue working until they're ready to migrate. A platform team changelog documents what changed between versions, and a migration guide explains how to upgrade.

Pipeline compliance checking — the platform can validate that service repositories use approved pipeline templates. A policy check scans repositories for CI/CD configurations and flags services that have custom pipelines not based on platform templates. This is governance at the pipeline level — ensuring security scans and approval gates aren't bypassed by teams writing their own pipelines.

10 Infrastructure Template Automation

Infrastructure template automation extends the template concept from CI/CD pipelines to cloud resources — databases, queues, storage, networking, and compute. Instead of teams writing Terraform from scratch to provision their infrastructure, they use platform-provided Terraform modules and self-service workflows that

produce compliant infrastructure automatically.

The platform team maintains a library of reusable Terraform modules that implement organizational standards for each resource type. These modules encode best practices: encryption by default, proper tagging for cost allocation, appropriate security group configurations, backup policies, monitoring integrations. A team using the module gets all of these by default without needing to know they need to specify them.

A platform Terraform module for ML infrastructure:

hcl

89 lines

```
# modules/ml-training-environment/main.tf

variable "environment" {
  type          = string
  description    = "Environment name (development, staging, production)"

  validation {
    condition     = contains(["development", "staging", "production"], var.environment)
    error_message = "Environment must be development, staging, or production."
  }
}

variable "team_name" {
  type          = string
  description    = "Team name for resource ownership and cost allocation"
}

variable "gpu_count" {
  type          = number
  description    = "Number of GPUs for training instances"
  default       = 1
}

variable "instance_type" {
  type          = string
  description    = "EC2 instance type"
  default       = "g4dn.xlarge"

  validation {
    condition     = can(regex("^[gp][0-9]+", var.instance_type))
    error_message = "Instance type must be a GPU instance (g or p family)."
  }
}

# The module creates all required resources with correct configuration
resource "aws_eks_node_group" "training" {
  cluster_name      = data.aws_eks_cluster.main.name
  node_group_name   = "${var.team_name}-${var.environment}-training"
```

```

# Mandatory tagging for cost allocation - not optional
tags = merge(local.mandatory_tags, {
    Environment = var.environment
    Team        = var.team_name
    Purpose     = "ml-training"
    ManagedBy   = "platform-terraform"
})

# Platform-enforced: always use encryption
disk_size = 200

scaling_config {
    min_size      = 0    # Can scale to zero when not training
    max_size      = 20
    desired_size  = 0
}

instance_types = [var.instance_type]

# Platform-enforced: mandatory lifecycle hooks for safe teardown
lifecycle {
    ignore_changes = [scaling_config[0].desired_size]
}
}

# Platform-enforced: always create a budget alert
resource "aws_budgets_budget" "training_budget" {
    name          = "${var.team_name}-training-budget"
    budget_type   = "COST"
    limit_amount  = local.budget_by_environment[var.environment]
    limit_unit    = "USD"
    time_unit     = "MONTHLY"

    notification {
        comparison_operator      = "GREATER_THAN"
        threshold                 = 80
        threshold_type            = "PERCENTAGE"
        notification_type         = "ACTUAL"
        subscriber_email_addresses = [data.team_contact.main.email]
    }
}

# Platform-enforced: CloudWatch log group with retention
resource "aws_cloudwatch_log_group" "training_logs" {
    name          = "/ml-training/${var.team_name}/${var.environment}"
    retention_in_days = local.retention_by_environment[var.environment]
    kms_key_id    = data.aws_kms_key.platform.arn
    tags          = local.mandatory_tags
}

```

Teams use this module in a simple configuration:

hcl

10 lines

```
# In team's infrastructure repository
module "training_environment" {
  source = "git::https://github.com/myorg/platform-terraform//modules/ml-training-environment?ref=v2.1.0"

  environment    = "production"
  team_name      = "fraud-detection"
  gpu_count      = 2
  instance_type  = "p3.2xlarge"
}
```

The team specifies four values. The module handles mandatory tagging, budget alerts, CloudWatch log groups, security group configuration, IAM roles, encryption settings, and everything else that the platform team has determined is required.

Self-service through Backstage — infrastructure can be self-provisioned through the developer portal. A Backstage template form asks: what type of infrastructure (database, queue, training environment), what environment, what size, what team. The scaffolder runs Terraform using the appropriate module, stores the state, and registers the resource in the catalog. The engineer never touches Terraform code — they fill out a form and get infrastructure.

Infrastructure teardown — self-service provisioning must be paired with self-service teardown and automatic cleanup. Environments that are no longer needed should be easily destroyable. Development environments should be automatically cleaned up after a period of inactivity. This prevents infrastructure sprawl and uncontrolled costs.

11 Kubernetes Resource Templates

Kubernetes resource templates provide standardized, pre-configured Kubernetes manifests for common workload patterns. Rather than each team writing their own Deployment, Service, HPA, PodDisruptionBudget, and ServiceMonitor YAML from scratch (and making slightly different choices each time), platform-provided templates encode the correct configuration for each workload type.

The templates enforce organizational standards — resource requests and limits must be set, liveness and readiness probes must be configured, security contexts must run as non-root, labels must include required fields for cost allocation and ownership. Teams configure what's unique to their service; everything else comes correctly configured from the template.

Helm chart as Kubernetes template — for ML model serving, the platform provides an opinionated Helm chart:

yaml

69 lines

```
# Chart defaults - all represent platform best practices
# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "ml-serving.fullname" . }}
  labels:
```

```

    {{- include "ml-serving.labels" . | nindent 4 }}
    # Mandatory labels for cost allocation and ownership
    platform.company.com/team: {{ required "team is required" .Values.team }}
    platform.company.com/cost-center: {{ required "costCenter is required" .Values.costCenter }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      {{- include "ml-serving.selectorLabels" . | nindent 6 }}
template:
  spec:
    # Platform-enforced: non-root execution
    securityContext:
      runAsNonRoot: true
      runAsUser: 1001
      fsGroup: 1001
      seccompProfile:
        type: RuntimeDefault

    containers:
      - name: {{ .Chart.Name }}
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag | required "image.tag is required" }}"

        # Platform-enforced: resource requests and limits are required
        resources:
          requests:
            memory: {{ required "resources.requests.memory is required" .Values.resources.requests.memory }}
            cpu: {{ required "resources.requests.cpu is required" .Values.resources.requests.cpu }}
          limits:
            memory: {{ .Values.resources.limits.memory | default (mul 2 .Values.resources.requests.memory) }}
            cpu: {{ .Values.resources.limits.cpu | default (mul 4 .Values.resources.requests.cpu) }}

        # Platform-enforced: probes required for all services
        livenessProbe:
          httpGet:
            path: /health
            port: http
          initialDelaySeconds: {{ .Values.probes.liveness.initialDelaySeconds | default 30 }}
          periodSeconds: 30
          failureThreshold: 3

        readinessProbe:
          httpGet:
            path: /ready
            port: http
          periodSeconds: 10
          failureThreshold: 3

        startupProbe:
          httpGet:
            path: /health
            port: http
          failureThreshold: {{ .Values.probes.startup.failureThreshold | default 60 }}
          periodSeconds: 10

        # Platform-enforced: read-only root filesystem
        securityContext:
          readOnlyRootFilesystem: true
          allowPrivilegeEscalation: false
          capabilities:
            drop: [ALL]

```

Kustomize component library — the platform can provide a library of Kustomize components that teams compose:

yaml

12 lines

```
# base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

components:
- github.com/myorg/platform-components//components/standard-monitoring?ref=v1.0
- github.com/myorg/platform-components//components/network-policy-default?ref=v1.0

resources:
- deployment.yaml
- service.yaml
```

The `standard-monitoring` component automatically adds a ServiceMonitor for Prometheus scraping and standard alerting rules. The `network-policy-default` component adds baseline network policy allowing ingress from the gateway and egress to required services.

Template validation with OPA — after rendering templates, platform policies validate the output before deployment:

rego

17 lines

```
# policy: all-deployments-must-have-resource-limits.rego
package kubernetes.admission

deny[msg] {
  input.request.kind.kind == "Deployment"
  container := input.request.object.spec.template.spec.containers[_]
  not container.resources.limits
  msg := sprintf("Container %v in Deployment %v must have resource limits defined",
    [container.name, input.request.object.metadata.name])
}

deny[msg] {
  input.request.kind.kind == "Deployment"
  not input.request.object.metadata.labels["platform.company.com/team"]
  msg := "Deployment must have platform.company.com/team label"
}
```

This policy, enforced by OPA Gatekeeper as a Kubernetes admission controller, prevents any Deployment from being created without resource limits or mandatory labels — regardless of how it was created (via Helm, kubectl, ArgoCD, or any other method).

Policy as Code Concepts

We covered OPA in the Security section as a security tool. Here we look at Policy as Code in the broader platform engineering context — as the mechanism for enforcing organizational standards automatically, making governance a property of the platform rather than a manual process.

Policy as Code means organizational rules about how software should be built and operated are written as code, versioned in Git, tested, and automatically enforced. The rules cover security requirements, cost controls, compliance mandates, operational standards, and quality gates — anything the organization requires of its software.

The transformative aspect of Policy as Code is the shift from audit-based governance (build software however you want, then check compliance afterward) to enforcement-based governance (standards are enforced at provisioning time, non-compliant configurations are rejected before they reach production). This is shift-left for governance — catching compliance violations at the point of creation rather than in a quarterly audit.

Layers of policy enforcement in a platform:

IDE plugins give developers real-time feedback as they write code. A Kubernetes manifest being edited shows an inline warning if resource limits are missing. A Terraform file shows an error if encryption is not enabled. Policies enforced at write time have zero impact on deployment velocity and give developers immediate feedback.

Pre-commit hooks enforce policies before code is committed. A pre-commit hook runs OPA or Conftest against all changed Terraform and Kubernetes files and blocks the commit if policy violations are found.

CI pipeline gates run policies against the full configuration during every CI run. A CI stage runs `conftest verify` against all rendered manifests and fails the pipeline if violations exist.

Kubernetes admission controllers enforce policies at cluster level. OPA Gatekeeper intercepts every Kubernetes API request and evaluates it against ConstraintTemplates. Non-compliant resources are rejected by the cluster, regardless of how they were applied.

Continuous compliance scanning checks deployed resources against policies on a schedule, detecting drift from compliant configurations.

Writing policies with OPA/Rego:

rego

40 lines

```
# Policy: ML model serving deployments must have GPU limits if they request GPUs
package platform.ml

import future.keywords.in

# List of GPU resource types
gpu_resources := {
  "nvidia.com/gpu",
  "amd.com/gpu",
}
```

```

violation[{"msg": msg}] {
    input.review.kind.kind = "Deployment"
    container := input.review.object.spec.template.spec.containers[_]

    # Container requests a GPU
    resource_key in object.keys(container.resources.requests)
    resource_key in gpu_resources

    # But doesn't set a limit for it
    not container.resources.limits[resource_key]

    msg := sprintf(
        "Container '%v' requests GPU resource '%v' but must also set a limit",
        [container.name, resource_key]
    )
}

# Policy: Production deployments must have at least 2 replicas
violation[{"msg": msg}] {
    input.review.kind.kind = "Deployment"
    input.review.object.metadata.namespace = "ml-production"
    replicas := input.review.object.spec.replicas
    replicas < 2
    msg := sprintf(
        "Production deployment '%v' must have at least 2 replicas, has %v",
        [input.review.object.metadata.name, replicas]
    )
}

```

Policy testing — policies should have unit tests:

rego

45 lines

```

# policy_test.rego
package platform.ml

test_gpu_request_without_limit_denied {
    count(violation) > 0 with input as {
        "review": {
            "kind": {"kind": "Deployment"},
            "object": {
                "metadata": {"namespace": "ml-production"},
                "spec": {
                    "replicas": 3,
                    "template": {"spec": {"containers": [{
                        "name": "model-server",
                        "resources": {

```

```

        "requests": {"nvidia.com/gpu": "1"},
        "limits": {} # Missing GPU limit
    }
    ]]]}
    }
    }
    }
    }

test_valid_gpu_deployment_allowed {
    count(violation) = 0 with input as {
        "review": {
            "kind": {"kind": "Deployment"},
            "object": {
                "metadata": {"namespace": "ml-production"},
                "spec": {
                    "replicas": 3,
                    "template": {"spec": {"containers": [{
                        "name": "model-server",
                        "resources": {
                            "requests": {"nvidia.com/gpu": "1"},
                            "limits": {"nvidia.com/gpu": "1"} # Limit set
                        }
                    ]}}
                }
            }
        }
    }
}

```

Policies tested with unit tests can be developed and refined with confidence, and the test suite documents exactly what each policy does and doesn't allow.

13 Standardization & Governance

Standardization and governance are the organizational outcomes that policy as code, golden paths, and templates collectively produce. They're worth discussing explicitly because the goal of the platform isn't just technical consistency — it's organizational capability and risk management.

Standardization means that common patterns across the organization are implemented consistently. All model serving services are deployed using the same Helm chart structure. All CI/CD pipelines follow the same security scan and approval workflow. All Kubernetes resources have the same required labels. All secrets are managed through the same secrets management system. Standardization makes the organization legible — any engineer can navigate any service's infrastructure because it follows the same patterns they know from other services.

The benefits of standardization compound over time. When you need to update a security configuration across all services — because a new vulnerability requires a change to every service's network policy — standardization means one change to one template, not a hundred changes to a hundred different configurations. When a new engineer joins and needs to understand how a service is deployed, standardization means the deployment of the service they're onboarding to works the same way as every other service they've seen.

Governance is the set of controls that ensure software and infrastructure meet organizational requirements — security policies, compliance mandates, cost controls, quality standards. Good governance is invisible when things are working correctly: developers build and deploy freely, and everything they build automatically meets requirements because the platform enforces them. Bad governance is a friction-generating process: every change requires approval, every deployment goes through a review gate, every new service requires paperwork.

The platform engineering approach to governance is to embed it in automation. Instead of requiring developers to fill out a security review form for each deployment, enforce security requirements at the CI pipeline and cluster admission level. Instead of requiring cost approval for every new resource, implement budget alerts and automatic cleanup policies that prevent cost overruns while allowing teams to self-provision.

The governance paradox — the most effective governance is the kind that developers don't notice. If developers are constantly working around governance controls because they slow things down, governance is failing — the controls are generating friction without producing compliance. If developers never think about governance because the platform handles it automatically, governance is succeeding — compliance is high and developer velocity is unaffected.

Escalation paths — not every situation fits the standard path. Good governance includes clear, fast escalation paths for exceptions. A team with a genuinely unusual requirement can request an exemption, which goes through a defined review process, gets documented, and is time-limited. The exception process should be fast enough that teams don't bypass governance to avoid waiting — if waiting for an exception takes 3 weeks, teams will just find workarounds. If it takes 2 business days, teams will use the proper process.

Software Bill of Materials at organizational scale — governance for the platform means knowing what's running everywhere. Maintaining catalog entries for all services, tracking dependencies, knowing which services use which libraries, understanding the blast radius of a vulnerability — this organizational awareness is a governance capability that good platform tooling enables.

14 Platform API Design Concepts

A platform isn't just a collection of tools — it's a product with an API. Everything a developer can do through the platform should be accessible through a well-designed API: provisioning infrastructure, creating pipelines, deploying services, querying service status, managing secrets. The API is the foundation that enables the portal, the CLI, programmatic access, and future tooling you haven't built yet.

Why a platform API matters — without a unified API, every platform capability has its own interface, its own authentication, its own data model. The Backstage template calls the GitHub API. The infrastructure provisioner calls Terraform CLI. The deployment system calls kubectl. Each interface is different. Building the CLI that ties these together requires integrating with a half-dozen different APIs. The platform API creates a unified abstraction layer.

API-first platform design — design the platform API before building the UI. The UI is an API client like any other. If the UI can do something, the API should be able to do it too. This ensures that programmatic access is a first-class capability, not an afterthought.

Platform API design principles:

Resource-oriented design — model platform capabilities as resources with standard CRUD operations. A service deployment is a resource. A database instance is a resource. A CI/CD pipeline run is a resource. Standard REST patterns (`GET /services/{id}` , `POST /services` , `PUT /services/{id}`) apply consistently.

Asynchronous operations — platform operations (creating infrastructure, running pipelines, deploying services) take time. The API should be asynchronous by default: POST to start an operation, receive an operation ID, poll the operation ID for status. Long-lived connections via WebSockets or SSE can stream operation progress.

diagram

22 lines

```
POST /v1/services
{ "name": "fraud-detection", "template": "ml-serving", "team": "ml-platform" }

→ 202 Accepted
{
  "operationId": "op-a8f3b291",
  "status": "in_progress",
  "estimatedDuration": "5m",
  "statusUrl": "/v1/operations/op-a8f3b291"
}

GET /v1/operations/op-a8f3b291
→ 200 OK
{
  "status": "completed",
  "result": {
    "serviceId": "svc-9c7d2e11",
    "repositoryUrl": "https://github.com/myorg/fraud-detection",
    "dashboardUrl": "https://backstage.company.com/catalog/fraud-detection"
  }
}
```

Versioned and stable — platform API clients (the portal, the CLI, automation scripts) break when the API changes incompatibly. Version the API from day one. Maintain at least one previous version when releasing breaking changes. Give clients a migration path and timeline.

Comprehensive error responses — API errors should include enough information to understand what went wrong and how to fix it:

```
{
  "error": {
    "code": "RESOURCE_QUOTA_EXCEEDED",
    "message": "GPU quota exceeded for team ml-platform in production",
    "details": {
      "requested": 8,
      "available": 4,
      "teamQuota": 16,
      "currentUsage": 12
    },
    "links": {
      "requestIncrease": "https://platform.company.com/quotas/request",
      "currentUsage": "https://platform.company.com/quotas/ml-platform"
    }
  }
}
```

Authentication and authorization — the platform API must integrate with your organization's SSO for authentication. Authorization should be fine-grained — a team should be able to manage their own services, query other teams' services read-only, and have no access to platform administration. Scoped API tokens for service account access, OIDC for human access.

SDK generation — from an OpenAPI specification, generate client SDKs in the languages your teams use (Python for ML engineers, TypeScript for frontend teams, Go for infrastructure engineers). Generated SDKs reduce integration friction and ensure client code stays current when the API evolves.

15 Scaling Engineering Teams with IDP

An internal developer platform isn't just about developer experience or operational efficiency — it's a fundamental enabler of organizational scaling. Understanding how IDPs enable engineering organizations to grow productively is what justifies the investment in platform engineering.

The problem with scaling engineering organizations without a platform is that complexity grows non-linearly. Doubling the number of engineering teams more than doubles the amount of infrastructure, the number of different configurations, the surface area for security vulnerabilities, the complexity of onboarding new engineers, and the coordination overhead. Without a platform, adding engineers makes the organization slower because the infrastructure complexity they generate exceeds the productive capacity they contribute.

Reducing time-to-productivity for new engineers is one of the most measurable impacts of a mature platform. In organizations without a platform, a new engineer's first task — deploying a simple change to their service — might take their first week because they need to understand the CI/CD system, get access to various tools, figure out how to configure Kubernetes, understand the monitoring setup, and learn the deployment process. With a mature platform and golden paths, that first deployment happens in the first day, and the onboarding time to full productivity is measured in days, not weeks.

The team topology effect — a good platform enables a team topology where team-to-platform-team ratios can be high (20:1 or more). The platform team provides capabilities, and each product team operates largely independently using those capabilities. Without the platform, operations teams have lower ratios (5:1 or lower) because they handle individual requests from development teams. The platform's self-service model is what makes the high ratio possible — developers provision what they need without platform team involvement.

Cognitive load management — Team Topologies (a framework for organizing engineering teams) identifies cognitive load as one of the primary constraints on team effectiveness. Teams have a limited cognitive budget. Every domain they need to understand consumes part of that budget. Infrastructure knowledge consumed by a product team displaces domain knowledge. The platform reduces the infrastructure cognitive load on product teams by abstracting it away, freeing cognitive capacity for the domain problems they exist to solve.

Consistency enables mobility — when all services follow the same patterns, engineers can move between teams (permanently or temporarily) without a long ramp-up on the new team's specific infrastructure setup. An ML engineer moving from the fraud detection team to the recommendations team knows immediately how their new service is deployed, monitored, and operated because it uses the same platform patterns as their previous service. This reduces the friction of team reorganizations and temporary assignments.

Standardization reduces incidents — a significant fraction of production incidents are caused by misconfigurations. Resource limits missing, health checks misconfigured, security groups too permissive, monitoring not set up. A platform that enforces correct configuration through templates and policies reduces this class of incident. The platform team's investment in getting the template right once prevents the same mistake being made across many services.

The platform flywheel — the more teams use the platform, the more feedback the platform team gets about what's working and what isn't. The more the platform is improved, the more teams want to use it. Teams that use the platform deploy faster, have fewer incidents, and spend less time on infrastructure. This success attracts more teams. The platform grows in adoption and capability, creating a virtuous cycle that compounds over time.

16 Measuring Platform Success Metrics

Platform teams are making significant investments — engineering time, infrastructure costs, organizational change. Justifying that investment and directing future investment to the highest-impact areas requires measuring outcomes. The right metrics for platform success align directly with the platform's purpose: reducing the friction, cost, and time associated with building and operating software.

Metrics for platform engineering fall into three categories: developer experience metrics (how engineers feel about the platform and how productive it makes them), outcome metrics (what the platform enables in terms of delivery speed and reliability), and platform health metrics (how well the platform itself is operating).

Developer experience metrics:

Developer satisfaction score (DevEx score) — a quarterly survey measuring developers' satisfaction with the tools, workflows, and processes they use. A single question like "On a scale of 1-10, how satisfied are you with your development environment and tools?" provides a comparable number over time. More detailed surveys break down satisfaction by specific areas: CI/CD, deployment, monitoring, documentation, self-service capabilities.

Time-to-first-deployment for new hires — measure from the day an engineer joins to the day they successfully deploy their first change to staging. This is a powerful metric because it captures the entire onboarding experience holistically. A good platform should achieve first deployment within the first day or two. Improvement in this metric directly represents reduced onboarding friction.

Time-to-provision new infrastructure — measure from when an engineer submits a self-service request to when the infrastructure is ready to use. For development environments, this should be minutes. For production infrastructure with approval workflows, it should be hours, not days.

Ticket volume to platform team — measure the number of tickets asking the platform team for help or to fulfill requests. A maturing platform should see this decrease over time as self-service capabilities improve. A spike in tickets indicates a gap in self-service coverage or documentation.

Outcome metrics:

Deployment frequency per team — how often do teams deploy to production? A platform that makes deployment easy should correlate with higher deployment frequency. Track over time and compare before and after platform improvements.

Lead time for changes — from when code is committed to when it's in production. Platform improvements to CI/CD speed, deployment automation, and approval workflows should reduce this.

Change failure rate — what percentage of production deployments cause incidents or rollbacks? Platform-enforced quality gates (testing, scanning, gradual rollouts) should reduce this over time.

Mean time to recovery — when incidents occur, how long does it take to restore service? Standardized runbooks, one-click rollback, and clear operational dashboards enabled by the platform reduce MTTR.

Platform health metrics:

Platform availability — uptime of the platform's own services (Backstage, ArgoCD, the CI/CD platform). A platform that's frequently unavailable blocks development work across the entire organization.

Template adoption rate — what percentage of services are using platform templates vs custom configurations? High adoption indicates the templates are meeting teams' needs. Low adoption indicates teams are finding the templates inadequate and building their own solutions.

Policy compliance rate — what percentage of deployed services comply with organizational policies? Improvement in this metric indicates the platform is successfully encoding governance.

Self-service fulfillment rate — what percentage of infrastructure requests are fulfilled through self-service (without platform team involvement)? A mature platform should have >90% self-service fulfillment.

Connecting to business metrics:

The ultimate justification for platform investment is business value. Calculate the engineering hours saved: if the platform saves each of 200 engineers 3 hours per week, that's 600 engineer-hours per week — the equivalent of 15 additional engineers. At \$200k/engineer/year, that's \$3M annual value from 3-4 platform engineers. Present this calculation to leadership, using actual measured time savings from developer surveys, to make the case for continued investment in platform capabilities.

Track separately the incidents prevented — if security policies caught 15 potential vulnerabilities before deployment, and each prevented incident would have cost 20 engineer-hours to respond to, that's 300 engineer-hours of incident response avoided. These prevention metrics are harder to measure but represent real value.

The metrics that matter most will differ by organization and by the platform's maturity stage. Early platforms should focus on adoption and developer satisfaction — is anyone using the platform, and do they like it? Mature platforms should focus on business outcomes — is the platform measurably accelerating delivery and reducing operational risk? Tracking the right metrics at each stage guides investment priorities and communicates platform value to stakeholders who control engineering budgets.

The thread connecting all of them is this: an internal developer platform is fundamentally a product whose customers are engineers and whose purpose is to reduce the cognitive load, waiting time, and undifferentiated infrastructure work that prevents engineers from spending their time on the problems they exist to solve. Every concept here — from golden paths to policy as code to success metrics — is a specific answer to the question of how to build a platform that makes engineers measurably more productive while making the organization more secure, consistent, and resilient.